DESIGNED BY:

MRIDUL GUPTA Ref: AI

Chapters

Chapter 1: Introduction to Java

What is Java?

Definition

Java is a **high-level, object-oriented, class-based, platform-independent programming language** used to develop desktop applications, web applications, mobile applications, enterprise software, games, cloud applications, and many other types of software.

Java was developed by **James Gosling** at **Sun Microsystems** and officially released in **1995**. Today, Java is maintained by **Oracle Corporation**.

Java follows the famous principle:

Write Once, Run Anywhere (WORA)

This means that Java programs can run on different operating systems without changing the source code, as long as a compatible Java Virtual Machine (JVM) is installed.

Explanation

A programming language is used to give instructions to a computer.

Just as humans communicate using languages like English or Hindi, programmers communicate with computers using programming languages such as Java.

Java is designed to be:

- Easy to learn
- Easy to read
- Secure
- Reliable
- Portable
- Suitable for both beginners and professionals

Unlike low-level languages, Java uses English-like syntax, making it easier to understand.

Real-Life Example

Imagine you write a letter in English.

Anyone who understands English can read it.

Similarly:

- You write Java code once.
- The JVM translates it for each operating system.
- The same Java program works on Windows, Linux, and macOS.

Java Example

```
public class Main {
  public static void main(String[] args) {
    System.out.println("Welcome to Java");
  }
}
```

Output

```
Welcome to Java
```

Interview Question

Q: What is Java?

Answer:

Java is a high-level, object-oriented, platform-independent programming language that follows the principle "Write Once, Run Anywhere" and is used for developing various types of software applications.

History of Java

Definition

The history of Java describes how the language was created, developed, and improved over time.

Explanation

In **1991**, a team of engineers at Sun Microsystems started a project called **Green Project**.
The project leader was **James Gosling**.
The goal was to create a programming language for electronic devices such as televisions and set-top boxes.
Initially, the language was named **Oak**, after an oak tree outside James Gosling's office.
Later, it was renamed **Java** because the name "Oak" was already registered as a trademark.
Java was officially released in **1995**.
In **2009**, **Oracle Corporation** acquired **Sun Microsystems** and became the maintainer of Java.

Timeline

Year	Event
1991	Green Project started
1991	Oak language created
1995	Java officially released
1996	JDK 1.0 released
2009	Oracle acquired Sun Microsystems
Present	Java is widely used around the world

Interview Question

Q: Who invented Java?

Answer:

Java was developed by James Gosling at Sun Microsystems in 1991 and officially released in 1995.

Features of Java

A **feature** is a special characteristic that makes Java useful and different from other programming languages.

1. High-Level Language

Definition

A high-level language is a programming language that is easy for humans to read and write.

Explanation

Java uses English-like words instead of binary numbers.

Example

```
int age = 20;
```

2. Platform Independent: Platform independence means the same Java program can run on different operating systems without changing the code.

Explanation

Java source code is first compiled into **bytecode**, which is then executed by the JVM on each operating system.

Example

A Java program compiled on Windows can also run on Linux and macOS if they have a JVM.

3. Object-Oriented

Definition

Java is an object-oriented programming (OOP) language, meaning programs are built using **classes** and **objects**.

Explanation

OOP helps organize code into reusable and maintainable components.

Real-Life Example

A **Car** has:

Properties

- Brand
- Color
- Speed

Behaviors

- Start
- Stop
- Brake

A Java class models these properties and behaviors.

4. Simple

Definition

Java is considered simple because it removes many complex features found in older languages such as C++.

Explanation

Java has:

- Automatic memory management
- No pointer arithmetic
- Clear syntax

This makes it easier to learn and reduces programming errors.

5. Secure

Definition

Java is designed with security features that help protect programs from malicious code.

Explanation

Java provides:

- Bytecode verification
- No direct memory access
- Security Manager (legacy)
- Automatic memory management

These features reduce many common security problems.

6. Robust

Definition

A robust language is one that is reliable and can handle errors effectively.

Explanation

Java is robust because it includes:

- Exception handling
- Strong type checking
- Garbage collection

These features help reduce crashes and improve program stability.

7. Portable

Definition

Portable means Java programs can be moved from one system to another with little or no modification.

Explanation

The JVM ensures Java programs behave consistently across platforms.

8. Multithreaded

Definition

Multithreading is the ability to execute multiple tasks simultaneously within the same program.

Real-Life Example

While downloading a file, you can still listen to music and browse the internet.
Java supports this kind of concurrent execution.

9. Distributed

Definition

Java supports distributed computing, allowing applications to communicate over networks.

Example

Online banking systems where servers and clients exchange data.

10. Dynamic

Definition

Java can load classes and objects during program execution, making it flexible and adaptable.

Applications of Java

Java is used in many industries.

Desktop Applications

Examples:

- Calculator
- Notepad
- Media Player

Web Applications

Examples:

- Online shopping websites
- Banking portals

- Hospital management systems

Mobile Applications
Many Android applications are developed using Java (alongside other languages in modern Android development).

Enterprise Applications
Large business software such as:
- Banking systems
- Airline reservation systems
- ERP software

Cloud Computing
Java is widely used for cloud-based services and backend systems.

Scientific Applications
Used for simulations, research tools, and data analysis.

Gaming
Java can be used to develop 2D and some 3D games.

Java Editions
Java is available in different editions based on development needs.

Java SE (Standard Edition)
Definition
Java SE provides the core Java libraries used for general-purpose programming.
Used For
- Desktop applications
- Console applications
- Learning Java

Java EE (Enterprise Edition)
Definition
Java EE extends Java SE with technologies for large-scale enterprise and web applications.
Used For
- Banking systems
- E-commerce websites
- Enterprise applications

Java ME (Micro Edition)
Definition
Java ME is designed for embedded systems and small devices with limited resources.
Used For
- Embedded devices

- IoT devices
- Older mobile platforms

Practice Questions

1. What is Java? Explain its main characteristics.
2. What does **WORA** stand for?
3. Who developed Java, and when was it released?
4. Why is Java called a platform-independent language?
5. Explain any five features of Java with examples.
6. What is the difference between Java SE, Java EE, and Java ME?
7. List five real-world applications of Java.
8. What makes Java simple and secure?
9. Explain the meaning of a high-level programming language.
10. Why is Java considered an object-oriented programming language?

Chapter 2: Java Program Structure & Basic Syntax

What is Syntax?

Definition

Syntax is the set of rules that defines how Java code must be written. Every programming language has its own syntax, and if these rules are not followed, the program will produce syntax errors.

Explanation

Think of syntax as the **grammar of a programming language**.

Just as an English sentence must follow grammar rules, a Java program must follow Java syntax rules.

Real-Life Example

Correct English:

I am learning Java.

Incorrect English:

Learning Java I am.

Similarly,

Correct Java:

int age = 20;

Incorrect Java:

int = age 20;

The second example breaks Java syntax rules.

Java Program Structure

Definition

A **Java program structure** is the standard layout or organization of a Java program. It defines where packages, imports, classes, methods, and statements should be placed.

Basic Structure

```
// Comments

package packageName;     // Optional
import packageName.*;    // Optional
public class Main {
   public static void main(String[] args) {
      // Statements
   }
}
```

Explanation of Each Part

1. Comments

Used to explain code.

```
// This is a comment
```

2. Package

Groups related classes together.

```
package school;
```

3. Import
Allows the use of classes from other packages.
import java.util.Scanner;

4. Class
Every Java program must contain at least one class.
public class Main
The class is like a blueprint.

5. Main Method
public static void main(String[] args)
This is the starting point of every Java application.
When the program runs, the JVM first looks for this method.

6. Statements
Statements are instructions executed by Java.
Example:
System.out.println("Hello");

First Java Program

```
public class Main {
  public static void main(String[] args) {
     System.out.println("Welcome to Java");
   }
}
```

Output
Welcome to Java

Line-by-Line Explanation
Line 1
public class Main

- public → Accessible from anywhere.
- class → Defines a class.
- Main → Class name.

Line 2
{
Starts the class body.

Line 3
public static void main(String[] args)
Let's understand each keyword.
public
The method can be accessed by the JVM.

static
Allows the method to be called without creating an object.

void
The method does not return any value.

main
Special method name where execution begins.

String[] args
Stores command-line arguments.

Line 5
System.out.println("Welcome");
Displays text on the console.

Keywords

Definition
Keywords are reserved words that have predefined meanings in Java. They cannot be used as variable names, method names, or class names.
Example
public
class
static
void
int
if
else
return

Why Can't Keywords Be Used as Variable Names?
Incorrect
int class = 10;
The compiler becomes confused because class already has a predefined meaning.
Correct
int totalMarks = 10;

Common Java Keywords

Keyword	Purpose
class	Defines a class
public	Accessible everywhere
private	Accessible only within the class
protected	Accessible in the package and subclasses
static	Belongs to the class rather than an object

Keyword	Purpose
void	No return value
return	Returns a value from a method
if	Conditional statement
else	Alternative condition
switch	Multiple-choice selection
case	A branch inside a switch
break	Exits a loop or switch
continue	Skips the current loop iteration
new	Creates an object
this	Refers to the current object
super	Refers to the parent class
final	Prevents modification or inheritance
extends	Inherits from another class
implements	Implements an interface
interface	Declares an interface
abstract	Declares an abstract class or method
try	Starts exception handling
catch	Handles an exception
finally	Executes after try/catch
throw	Throws an exception
throws	Declares possible exceptions
package	Declares a package
import	Imports classes from packages

Note: Modern Java has **50+ reserved keywords**. You don't need to memorize them all immediately—learn them as you encounter them.

Identifiers

Definition

An **identifier** is the name given by the programmer to variables, methods, classes, objects, packages, or interfaces.

Examples

```
int age = 20;
String studentName = "Amit";
class Employee {
}
void calculateSalary() {
}
```

Here,

- age
- studentName
- Employee
- calculateSalary

are identifiers.

Rules for Identifiers

Rule 1

Must start with:

- Letter
- _
- $

Valid

age

_name

$salary

Rule 2

Cannot start with a number.

Wrong

1age

Correct

age1

Rule 3

Cannot contain spaces.

Wrong

student name

Correct

studentName

Rule 4

Cannot be a keyword.

Wrong

int class = 10;

Rule 5

Java is case-sensitive.

age

Age

AGE

These are three different identifiers.

Literals

Definition

A **literal** is a fixed value written directly in the source code.

Types of Literals

Integer Literal

100

250

-50

Floating-Point Literal

10.5

99.99

Character Literal

'A'

'Z'

String Literal

"Hello"

"Java"

Boolean Literal

true

false

Null Literal

null

Used when a reference variable points to no object.

Comments

Definition

Comments are notes added to the source code to explain what the code does. They are ignored by the Java compiler.

Types of Comments

Single-Line Comment

```
// Calculate salary
```

Multi-Line Comment

```
/*
This method
calculates salary
*/
```

Documentation Comment

```
/**
 * Returns the total salary.
 */
```

Used to generate API documentation with the javadoc tool.

Java Naming Conventions

Definition

Naming conventions are recommended rules for choosing meaningful names in Java. They improve readability and maintainability.

Class Names

Use **PascalCase.**

Student
EmployeeDetails
BankAccount

Variable Names

Use **camelCase.**

studentName
totalMarks
employeeSalary

Method Names

Use **camelCase.**

calculateSalary()
printReport()
getName()

Constant Names

Use **UPPER_CASE** with underscores.

MAX_SIZE
PI
MIN_MARKS

Package Names

Use **lowercase.**

com.school.management

Basic Output Statements

System.out.print()

Prints text without moving to the next line.

```
System.out.print("Hello ");
System.out.print("World");
```

Output:

Hello World

System.out.println()

Prints text and moves to the next line.

```
System.out.println("Hello");
System.out.println("World");
```

Output:

Hello
World

System.out.printf()

Prints formatted output.

```
String name = "Amit";
int age = 20;
System.out.printf("Name: %s, Age: %d", name, age);
```

Output:

Name: Amit, Age: 20

Practice Questions

1. What is syntax? Why is it important?
2. Explain the structure of a Java program.
3. What is the purpose of the main() method?
4. What are Java keywords? Give five examples.
5. What is an identifier? List its naming rules.
6. What are literals? Explain their types with examples.
7. What are the three types of comments in Java?
8. Explain Java naming conventions for classes, variables, methods, constants, and packages.
9. What is the difference between print(), println(), and printf()?
10. Why is Java called a case-sensitive language?

Chapter 3: Variables, Data Types & Type Casting

Variable

Definition

A **variable** is a named memory location used to store data. The value stored in a variable can change during program execution unless it is declared as final.

Explanation

When a program runs, it needs to store information such as:

- Student name
- Employee salary
- Product price
- User age

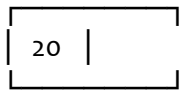
Variables allow the program to save and use this information.

Think of a variable as a **labeled box**.

- The label is the variable name.
- The box stores the value.

Example:

age



Syntax

```
dataType variableName = value;
```

Example

```
int age = 20;
String name = "Rahul";
double salary = 50000.75;
```

Real-Life Example

Imagine a school register.

Student	Marks
Amit	90
Neha	85

Here,
marks
is like a variable.

Why Do We Need Variables?

Without variables, every value would have to be written repeatedly.

Instead of

```
System.out.println(20);
System.out.println(20);
System.out.println(20);
```

we write

```
int age = 20;
System.out.println(age);
System.out.println(age);
System.out.println(age);
```

This makes programs easier to read and maintain.

Rules for Naming Variables

A variable name must follow Java's identifier rules.

Rule 1

Can start with

- Letter
- _
- $

Correct

age
_salary
$price

Rule 2

Cannot start with a number.

Wrong
1age
Correct
age1

Rule 3

Cannot contain spaces.

Wrong
student age
Correct
studentAge

Rule 4

Cannot use Java keywords.

Wrong

```
int class = 10;
```

Rule 5

Java is case-sensitive.

age
Age
AGE

These are three different variables.

Types of Variables

Java has three main types of variables.

1. Local Variable

Definition

A **local variable** is declared inside a method, constructor, or block. It can only be used within that block.

Example

```
public class Main {
  public static void main(String[] args) {
    int age = 20;
    System.out.println(age);
  }
}
```

age exists only inside the main() method.

2. Instance Variable

Definition

An **instance variable** is declared inside a class but outside all methods. Each object has its own copy.

Example

```
class Student {
  String name;
}
```

Every Student object has its own name.

3. Static Variable

Definition

A **static variable** belongs to the class rather than to individual objects. All objects share the same copy.

Example

```
class Student {
  static String school = "ABC School";
}
```

Every student uses the same school name.

Constant

Definition

A **constant** is a value that cannot be changed after it has been assigned. In Java, constants are created using the final keyword.

Example

final double PI = 3.14159;

Wrong

PI = 3.14;

This causes a compile-time error.

Why Use Constants?

Constants prevent accidental modification of fixed values.

Examples:

- PI
- GST rate
- Maximum marks
- Company name

Data Type

Definition

A **data type** specifies:

- what kind of value a variable can store,
- how much memory is allocated,
- and what operations can be performed on that value.

Example

```
int age = 20;
double salary = 55000.50;
char grade = 'A';
```

Each variable stores a different type of data.

Categories of Data Types

Java has two main categories.

Data Types

```
|
├── Primitive
|
└── Non-Primitive
```

Primitive Data Types

Definition

Primitive data types are built-in types that store simple values directly.

Java has **8 primitive data types**.

Type	Size	Stores
byte	1 byte	Small integers
short	2 bytes	Medium integers
int	4 bytes	Whole numbers
long	8 bytes	Large whole numbers
float	4 bytes	Decimal numbers
double	8 bytes	Large decimal numbers
char	2 bytes	Single Unicode character
boolean	JVM-dependent representation	true or false

byte

Definition

Stores very small integers.

Example

```
byte age = 25;
```

Range

-128 to 127

short

Stores larger integers than byte.

```
short population = 30000;
```

int
Most commonly used integer type.
int salary = 45000;

long
Stores very large integers.
long distance = 9000000000L;
The L suffix indicates a long literal.

float
Stores decimal numbers with single precision.
float price = 25.5f;
The f suffix is required because decimal literals are double by default.

double
Stores decimal numbers with higher precision than float.
double pi = 3.1415926535;

char
Stores a single character.
char grade = 'A';
A char uses single quotes.

boolean
Stores logical values.
boolean passed = true;
Only two values are allowed:
true
false

Non-Primitive Data Types

Definition
Non-primitive (reference) data types store references (addresses) to objects rather than the object data directly.
Examples include:

- String
- Arrays
- Classes
- Interfaces
- Enums
- Records (modern Java)

Example
String name = "Rahul";
Here, name refers to a String object.

Difference Between Primitive and Non-Primitive

Primitive	Non-Primitive
Built into Java	Created using classes or provided as library classes

Primitive	Non-Primitive
Stores actual value	Stores a reference to an object
Fixed size	Size varies
Cannot be null	Can be null
Examples: int, double, char	Examples: String, arrays, custom classes

Default Values

Instance and static variables receive default values if not initialized.

Data Type	Default Value
byte	0
short	0
int	0
long	0L
float	0.0f
double	0.0
char	'\u0000' (null character)
boolean	false
Reference Types	null

Important: Local variables do **not** get default values. They must be initialized before use.

Type Conversion

Definition

Type conversion is the process of changing a value from one data type to another.

There are two types:

1. Implicit (Automatic) Conversion
2. Explicit (Manual) Type Casting

Implicit Type Conversion (Widening)

Definition

When Java automatically converts a smaller data type into a larger compatible data type.

No data is lost.

Example

```
int number = 50;
double value = number;
```

Output

50.0

Explicit Type Casting (Narrowing)

Definition

When the programmer manually converts a larger data type into a smaller one.

Some information may be lost.

Example

```
double value = 45.89;
int number = (int) value;
```

Output

45
The decimal part is discarded.

Type Conversion Order

Java automatically converts data in this order:

byte
↓
short
↓
int
↓
long
↓
float
↓
double

This is called **widening conversion**.

Wrapper Classes (Introduction)

Definition

A **wrapper class** is an object representation of a primitive data type.

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Example

```
Integer age = 25;

Double salary = 55000.50;
```

Wrapper classes are useful when working with collections and generic classes.

Practice Questons

- What is a variable? Why is it used?
- Explain the rules for naming variables in Java.
- Differentiate between local, instance, and static variables.
- What is a constant? Why is the final keyword used?
- What is a data type? Why is it important?
- List all eight primitive data types with examples.
- What is the difference between primitive and non-primitive data types?
- What are default values? Which variables receive them?
- Explain implicit and explicit type conversion with examples.
- What is a wrapper class? Name the wrapper class for each primitive type.

Chapter 4: Operators in Java

What is an Operator?

Definition

An **operator** is a special symbol that performs a specific operation on one or more operands (values or variables) and produces a result.

Explanation

Operators are used to perform tasks such as:

- Addition
- Subtraction
- Comparison
- Logical decisions
- Assignment
- Bit manipulation

Without operators, Java programs would not be able to calculate values or make decisions.

Example

```
int a = 10;
int b = 20;
int sum = a + b;
```

Here:

- + ⟶ Operator
- a and b ⟶ Operands
- sum ⟶ Result

Real-Life Example

Suppose you have:

- 5 apples
- 3 apples

Using the **+** operator:

5 + 3 = 8

The + operator performs the addition.

Types of Operators in Java

Java provides the following categories of operators:

1. Arithmetic Operators
2. Unary Operators
3. Assignment Operators
4. Relational (Comparison) Operators
5. Logical Operators
6. Bitwise Operators
7. Shift Operators
8. Ternary Operator
9. instanceof Operator

Arithmetic Operators

Definition

Arithmetic operators are used to perform mathematical calculations on numeric values.

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus (Remainder)

Addition (+)
Definition
Adds two operands.
Example
```
int a = 10;
int b = 5;
System.out.println(a + b);
```
Output
15

Subtraction (-)
Subtracts one operand from another.
```
int a = 20;
int b = 8;
System.out.println(a - b);
```
Output
12

Multiplication (*)
Multiplies two numbers.
```
System.out.println(5 * 6);
```
Output
30

Division (/)
Divides one number by another.
```
System.out.println(20 / 5);
```
Output
4
Important
Integer division removes the decimal part.
```
System.out.println(10 / 3);
```
Output
3
To get a decimal result:
```
System.out.println(10.0 / 3);
```
Output
3.3333333

Modulus (%)

Definition

Returns the remainder after division.

Example

```
System.out.println(10 % 3);
```

Output

1

Another example

```
System.out.println(25 % 5);
```

Output

0

Real-Life Use of %

Checking whether a number is even.

```
int number = 18;
if(number % 2 == 0)
System.out.println("Even");
```

Unary Operators

Definition

A **unary operator** works on only **one operand**.

Operators

- +
- -
- ++
- --
- !

Unary Plus

Returns the value unchanged.

```
int a = 10;
System.out.println(+a);
```

Output

10

Unary Minus

Changes the sign.

```
int a = 10;
System.out.println(-a);
```

Output

-10

Increment (++)

Definition

Increases the value by **1**.

Example

```
int x = 5;
x++;
System.out.println(x);
```

Output

6

Decrement (--)

Definition

Decreases the value by **1.**

```
int x = 5;
x--;
System.out.println(x);
```

Output

4

Prefix Increment

```
int a = 5;
System.out.println(++a);
```

Output

6

The variable is incremented **before** it is used.

Postfix Increment

```
int a = 5;
System.out.println(a++);
```

Output

5

After execution

a = 6

The variable is used **before** it is incremented.

Prefix vs Postfix

```
int a = 10;
System.out.println(++a);
```

Output

11

```
int a = 10;
System.out.println(a++);
```

Output

10

Later

a = 11

Assignment Operators

Definition

Assignment operators assign values to variables and can combine assignment with another operation.

Simple Assignment

```
int x = 10;
```

Addition Assignment

x += 5;

Equivalent to

x = x + 5;

Subtraction Assignment

x -= 2;

Multiplication Assignment

x *= 3;

Division Assignment

x /= 2;

Modulus Assignment

x %= 3;

Table

Operator	Equivalent
+=	x = x + value
-=	x = x - value
*=	x = x * value
/=	x = x / value
%=	x = x % value

Relational Operators

Definition

Relational operators compare two values and return either **true** or **false.**

Operator	Meaning
==	Equal
!=	Not Equal
>	Greater Than
<	Less Than
>=	Greater Than or Equal
<=	Less Than or Equal

Example

```
int a = 10;
int b = 20;
System.out.println(a < b);
```

Output

true

Example

```
System.out.println(10 == 20);
```

Output

false

Logical Operators

Definition

Logical operators combine or invert boolean expressions.
These operators are commonly used in if, while, and for statements.

Logical AND (&&)

Returns **true** only if **both** conditions are true.
Truth Table

A	B	A && B
T	T	T
T	F	F
F	T	F
F	F	F

Example

```
int age = 20;
boolean citizen = true;
System.out.println(age >= 18 && citizen);
```

Output

true

Logical OR (||)

Returns **true** if **at least one** condition is true.
Truth Table

A	B	A || B
T	T	T
T	F	T
F	T	T
F	F	F

Example

```
System.out.println(false || true);
```

Output

true

Logical NOT (!)

Reverses the boolean value.

Example

```
boolean result = true;
System.out.println(!result);
```

Output

false

Short-Circuit Evaluation

&&

If the first condition is **false,** Java does **not** evaluate the second condition because the overall result cannot become true.

Example

```
int a = 5;
if(a > 10 && ++a > 6){
}
```

++a is **not** executed because a > 10 is already false.

||

If the first condition is **true**, Java skips the second condition because the overall result is already true.

Bitwise Operators

Definition

Bitwise operators perform operations on the individual **bits** of integer values.

Operator	Meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT

Example

```
int a = 5;
int b = 3;
System.out.println(a & b);
```

Bitwise operations are commonly used in low-level programming, device drivers, and performance-critical code.

Shift Operators

Definition

Shift operators move the bits of a number to the left or right.

Operator	Meaning
<<	Left Shift
>>	Signed Right Shift
>>>	Unsigned Right Shift

Example

```
System.out.println(5 << 1);
```

Output

10

Ternary Operator

Definition

The **ternary operator** is a shorthand way to write an if-else statement.

Syntax

```
condition ? value1 : value2;
```

Example

```
int age = 20;
String result = (age >= 18) ? "Adult" : "Minor";
```

Output

Adult

instanceof Operator

Definition

The instanceof operator checks whether an object belongs to a particular class or interface.

Example

```
Student s = new Student();
System.out.println(s instanceof Student);
```

Output

true

Operator Precedence

Definition

Operator precedence determines the order in which operators are evaluated in an expression.

Example

```
int result = 10 + 5 * 2;
```

Output

20

Why?

Because multiplication (*) has higher precedence than addition (+).

Common Precedence Order

1. ()
2. Unary (++, --, !)
3. *, /, %
4. +, -
5. Relational (<, >, <=, >=)
6. Equality (==, !=)
7. Logical AND (&&)
8. Logical OR (||)
9. Ternary (?:)
10. Assignment (=, +=, -=, etc.)

Common Mistakes

1. Using = instead of ==

❌ Wrong

```
if(a = 10)
```

☑ Correct

```
if(a == 10)
```

2. Forgetting f for float

❌

```
float x = 5.5;
```

☑

```
float x = 5.5f;
```

3. Confusing Prefix and Postfix Increment

```
int x = 5;
System.out.println(x++);
```

Output:

5

After execution:

x = 6

Practice Questions

- What is an operator?
- Explain all arithmetic operators with examples.
- What is the difference between ++i and i++?
- Explain the % operator with a real-world use case.
- What is the difference between = and ==?
- Explain logical &&, ||, and ! with truth tables.
- What is short-circuit evaluation?
- What is the purpose of the ternary operator?
- What does the instance of operator do?
- Explain operator precedence with an example.

Chapter 5: Control Statements (Decision Making & Loops)

What are Control Statements?

Definition

Control statements are statements that control the flow (order) of execution of a Java program. They decide **which statements should execute, when they should execute, and how many times they should execute**.

Without control statements, Java programs execute statements **sequentially**, from top to bottom.

Explanation

Suppose you are writing a program to determine whether a student has passed an exam.

- If the marks are **40 or above**, display **"Pass"**.
- Otherwise, display **"Fail"**.

The program needs to **make a decision**. This is done using control statements.

Similarly, if you want to print numbers from 1 to 100, you do not write System.out.println() 100 times. Instead, you use a **loop**, which is also a control statement.

Types of Control Statements

Java control statements are divided into three categories:

Control Statements
│
├── Decision-Making Statements
│ ├── if
│ ├── if-else
│ ├── else-if ladder
│ ├── Nested if
│ └── switch
│
├── Looping Statements
│ ├── for
│ ├── while
│ ├── do-while
│ └── Enhanced for
│
└── Jump Statements
├── break
├── continue
└── return

Part 1: Decision-Making Statements

if Statement

Definition

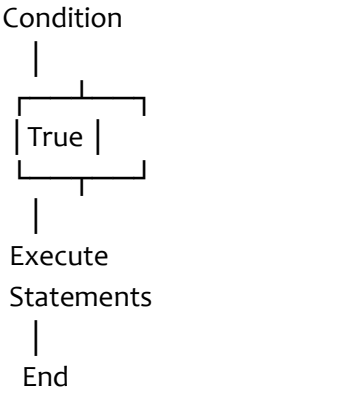
The **if statement** executes a block of code **only if** a specified condition evaluates to true.

If the condition is false, the block is skipped.

Syntax

```
if (condition) {
   // statements
}
```

Flow



False → Skip Statements

Example

```
int age = 20;
if (age >= 18) {
   System.out.println("Eligible to Vote");
}
```

Output

Eligible to Vote

Real-Life Example

Imagine entering a movie theater.

Rule:

Age ≥ 18

If true:

Entry Allowed

Otherwise:

No Entry

if-else Statement

Definition

The **if-else statement** chooses between **two blocks of code**.

- If the condition is true → execute the if block.
- Otherwise → execute the else block.

Syntax

```
if (condition) {
   // true block
} else {
   // false block
}
```

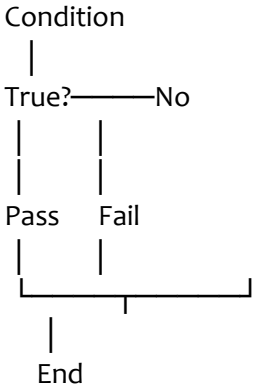
Example

```
int marks = 35;
if (marks >= 40) {
   System.out.println("Pass");
} else {
   System.out.println("Fail");
}
```

Output

Fail

Flow



else-if Ladder

Definition

The **else-if ladder** is used when there are **multiple conditions** to test.

Java checks each condition from top to bottom.

- The first true condition is executed.
- The remaining conditions are skipped.

Syntax

```
if(condition1){
}
else if(condition2){
}
else if(condition3){
}
else{
}
```

Example

```
int marks = 85;
if (marks >= 90) {
   System.out.println("Grade A");
}
else if (marks >= 80) {
   System.out.println("Grade B");
}
else if (marks >= 70) {
   System.out.println("Grade C");
}
else {
   System.out.println("Fail");
}
```

Output

Grade B

Real-Life Example
School grading system.
90–100 → Grade A
80–89 → Grade B
70–79 → Grade C
Below 40 → Fail

Nested if

Definition
A **nested if** is an if statement placed inside another if statement.
It is used when a second condition should be checked only after the first condition is true.

Example

```
int age = 25;
boolean citizen = true;
if(age >= 18){
   if(citizen){
      System.out.println("Eligible");
   }
}
```

Explanation
First check:
Age ≥ 18 ?
If yes,
then check:
Citizen?

switch Statement

Definition
The **switch statement** selects one block of code from multiple possible choices based on the value of an expression.
It is often simpler and more readable than using many else-if statements.

Syntax

```
switch(expression){
case value1:
   break;
case value2:
   break;
default:
}
```

Example

```
int day = 2;
switch(day){
case 1
   System.out.println("Monday");
   break;
case 2:
```

```
  System.out.println("Tuesday");
  break
case 3:
  System.out.println("Wednesday");
  break;
default:
  System.out.println("Invalid");
}
```

Output

Tuesday

What is break?

The break statement terminates the current case.

Without break, Java continues executing the next cases.

Example Without break

```
int day = 1;
switch(day){
case 1:
  System.out.println("Monday");
case 2:
  System.out.println("Tuesday");
case 3:
  System.out.println("Wednesday");
}
```

Output

Monday

Tuesday

Wednesday

This behavior is called **fall-through**.

Modern Switch Expression (Java 14+)

Java also supports a cleaner switch syntax.

```
String day = switch (2)
  case 1 -> "Monday";
  case 2 -> "Tuesday";
  default -> "Invalid";
};
System.out.println(day);
```

Part 2: Looping Statements

Loop

Definition

A **loop** repeatedly executes a block of code while a condition remains true.

Why Use Loops?

Without loops

```
System.out.println(1);
System.out.println(2)
```

```
System.out.println(3);
System.out.println(4);
System.out.println(5);
```
With loop
```
for(int i=1;i<=5;i++){
   System.out.println(i);
}
```
The loop is shorter, cleaner, and easier to maintain.

for Loop

Definition

A **for loop** is used when the number of iterations is known in advance.

Syntax

```
for(initialization; condition; update){
   statements
}
```

Example

```
for(int i=1;i<=5;i++){
   System.out.println(i);
}
```

Output

1

2

3

4

5

Parts of a for Loop

```
for(int i=1; i<=5; i++)
```

- **Initialization** → int i = 1
- **Condition** → i <= 5
- **Update** → i++

while Loop

Definition

The **while loop** executes a block of code as long as its condition is true.

It is useful when the number of iterations is **not known beforehand**.

Syntax

```
while(condition){
   statements;
}
```

Example

```
int i = 1;
while(i <= 5){
   System.out.println(i);
   i++;
}
```

do-while Loop

Definition

The **do-while loop** executes its body **at least once,** even if the condition is false, because the condition is checked **after** the loop body.

Syntax

```
do{
   statements;
}while(condition);
```

Example

```
int i = 1
do{
   System.out.println(i);
   i++;
}while(i<=5);
```

Difference Between while and do-while

while

Checks the condition first.

Condition

↓

Body

do-while

Executes the body first.

Body

↓

Condition

Enhanced for Loop (For-Each)

Definition

The **enhanced for loop** is used to iterate through arrays and collections without using an index.

Example

```
int[] numbers = {10,20,30};
for(int num : numbers){
   System.out.println(num);
}
```

Output

10

20

30

Part 3: Jump Statements

break

Definition

The **break** statement immediately terminates the current loop or switch statement.

Example

```
for(int i=1;i<=10;i++)
  if(i==5)
    break;
  System.out.println(i);
}
```

Output

1
2
3
4

continue

Definition

The **continue** statement skips the current iteration of a loop and proceeds with the next iteration.

Example

```
for(int i=1;i<=5;i++){
  if(i==3)
    continue;
  System.out.println(i);
}
```

Output

1
2
4
5

return

Definition

The **return** statement terminates a method and optionally returns a value to the caller.

Example

```
public static int square(int n){
  return n * n;
}
```

Comparison of Loops

Loop	Condition Checked	Executes at Least Once?	Best Used When
for	Before	No	Number of iterations is known
while	Before	No	Number of iterations is unknown
do-while	After	Yes	Body must execute at least once
Enhanced for	Before	No	Traversing arrays or collections

Common Mistakes

Infinite Loop

```
while(true){
}
```

Runs forever unless interrupted.

Missing Update

```
int i = 1;
while(i<=5){
   System.out.println(i);
}
```

i is never incremented, so the loop never ends.

Forgetting break in switch

This causes **fall-through**, where execution continues into subsequent cases unintentionally.

Practice Questions

- What are control statements?
- Explain the if statement with an example.
- What is the difference between if, if-else, and else-if?
- What is a nested if? When should it be used?
- Explain the switch statement. What is the purpose of break?
- What is fall-through in a switch statement?
- Compare for, while, and do-while loops.
- What is an enhanced for loop? Give an example.
- What is the difference between break, continue, and return?
- What is an infinite loop? How can it occur?

Chapter 6: Arrays in Java

Arrays are one of the most frequently asked topics in interviews and are used extensively in data structures and algorithms.

What is an Array?

Definition

An **array** is a collection of elements of the **same data type**, stored in **contiguous memory locations**, and accessed using an **index**.

In simple words, an array allows you to store **multiple values of the same type in a single variable**.

Why Do We Need Arrays?

Imagine you want to store the marks of 100 students.

Without Array

```
int mark1 = 85;
int mark2 = 90;
int mark3 = 76;
...
int mark100 = 88;
```

This is difficult to manage.

With Array

```
int[] marks = new int[100];
```

Now all 100 marks are stored in one variable.

Real-Life Example

Think about a classroom.

Instead of keeping 40 separate attendance sheets, you maintain **one attendance register** containing all students' names.

Similarly,

```
marks[]
```

stores all marks together.

Characteristics of Arrays

- Stores multiple values.
- All elements must have the **same data type**.
- Elements are stored in contiguous memory.
- Array size is **fixed** after creation.
- Each element is accessed using an index.
- Indexing starts from **0**.

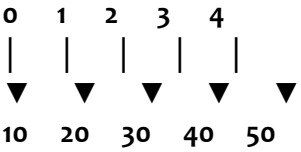
Memory Representation

Example

```
int[] numbers = {10,20,30,40,50};
```

Memory

Index



Notice:

First element → Index 0

Second element → Index 1

Last element → Index

length - 1

Declaration of Array

Definition

Declaration means telling Java that a variable will store an array.

Syntax

```
dataType[] arrayName;
```

Example

```
int[] numbers;
```

Another valid syntax

```
int numbers[];
```

The first style is preferred.

Array Creation

Definition

Creating an array means allocating memory for its elements.

Syntax

```
arrayName = new dataType[size];
```

Example

```
int[] numbers = new int[5];
```

Java creates space for 5 integers.

Array Initialization

Definition

Initialization means assigning values to array elements.

Method 1

```
int[] numbers = new int[5];
numbers[0] = 10;
numbers[1] = 20;
numbers[2] = 30;
numbers[3] = 40;
numbers[4] = 50;
```

Method 2

```
int[] numbers = {10,20,30,40,50};
```

This is the most common method.

Accessing Array Elements

Definition

Array elements are accessed using their index.

Syntax

```
arrayName[index]
```

Example

```
int[] numbers = {10,20,30};
System.out.println(numbers[0]);
System.out.println(numbers[2]);
```

Output

10

30

Modifying Array Elements

Arrays allow changing values.

Example

```
int[] numbers = {10,20,30};
numbers[1] = 50;
System.out.println(numbers[1]);
```

Output

50

Array Length

Definition

The length property returns the total number of elements in an array.

Example

```
int[] numbers = {10,20,30,40};
System.out.println(numbers.length);
```

Output

4

Default Values

When an array is created but not initialized, Java assigns default values.

Example

```
int[] numbers = new int[3];
```

Memory

0

0

0

Default Values

Type	Default Value
byte	0
short	0
int	0
long	0L
float	0.0f

Type	Default Value
double	0.0
char	'\u0000'
boolean	false
String	null

Traversing an Array

Definition

Traversing means visiting every element of the array.

Using for Loop

```
int[] numbers = {10,20,30,40}
for(int i=0;i<numbers.length;i++){
   System.out.println(numbers[i]);
}
```

Using Enhanced for Loop

```
int[] numbers = {10,20,30,40};
for(int num : numbers){
   System.out.println(num);
}
```

Difference

Normal for

- Uses index
- Can modify elements
- Can traverse forward or backward

Enhanced for

- Simpler
- No index
- Read-only access to elements

Types of Arrays

Java supports:

- One-Dimensional Array
- Two-Dimensional Array
- Multidimensional Array
- Jagged Array

One-Dimensional Array

Stores data in a single row.

Example

```
int[] numbers = {10,20,30};
```

Memory

10 20 30

Two-Dimensional Array

Definition

A two-dimensional array stores data in rows and columns.

Example

```
int[][] matrix = {
   {1,2,3},
   {4,5,6}
};
```

Memory

1 2 3

4 5 6

Access

```
System.out.println(matrix[1][2]);
```

Output

6

Multidimensional Array

Stores data in three or more dimensions.

Example

```
int[][][] cube = new int[2][3][4];
```

Used in scientific computing, graphics, simulations, and games.

Jagged Array

Definition

A jagged array is a two-dimensional array where each row can have a different number of columns.

Example

```
int[][] marks = {
   {90,80},
   {70,60,50},
   {100}
};
```

Memory

90 80

70 60 50

100

Passing Arrays to Methods

Example

```
public static void printArray(int[] arr){
   for(int num : arr)
      System.out.println(num);
}
```

Calling

```
int[] numbers = {10,20,30};
printArray(numbers);
```

Returning Arrays

Methods can also return arrays.

Example

```
public static int[] getNumbers(){
   return new int[]{1,2,3};
}
```

Arrays Utility Class

Java provides the Arrays class in the java.util package.

Useful methods:

Method	Purpose
sort()	Sort array
binarySearch()	Search
fill()	Fill array
copyOf()	Copy array
equals()	Compare arrays
toString()	Convert array to string

Example

```
import java.util.Arrays;
int[] arr = {4,2,8,1};
Arrays.sort(arr);
System.out.println(Arrays.toString(arr));
```

Output

[1, 2, 4, 8]

Common Errors

ArrayIndexOutOfBoundsException

Occurs when accessing an invalid index.

Example

```
int[] arr = {10,20,30};
System.out.println(arr[5]);
```

Output

Exception:
ArrayIndexOutOfBoundsException

NullPointerException

Occurs when the array reference is null.

Example

```
int[] arr = null;
System.out.println(arr.length);
```

Advantages of Arrays

- Easy to store multiple values.
- Fast access using index.
- Efficient memory usage.
- Simple to traverse.
- Foundation for many data structures.

Disadvantages of Arrays

- Fixed size.
- Can store only one data type.
- Insertion and deletion are costly.
- Cannot grow dynamically.

Difference Between Array and ArrayList

Array	ArrayList
Fixed size	Dynamic size
Stores primitives and objects	Stores objects (or wrapper classes)
Faster	Slightly slower
No built-in methods like add/remove	Rich API (add(), remove(), contains(), etc.)
Length accessed by length	Size accessed by size()

Practice Questions

- What is an array? Why is it used?
- Explain the characteristics of arrays.
- How do you declare, create, and initialize an array?
- Why does array indexing start from 0?
- What is the difference between a one-dimensional and a two-dimensional array?
- What is a jagged array? Give an example.
- Explain the difference between a normal for loop and an enhanced for loop for arrays.
- What is the purpose of the length property?
- What is ArrayIndexOutOfBoundsException, and how can you avoid it?
- Compare an array and an ArrayList.

Chapter 7: Methods in Java

What is a Method?

Definition

A **method** is a **named block of code** that performs a specific task. It can be called (invoked) whenever needed, allowing the same code to be reused without rewriting it.

A method may:

- Accept input (parameters)
- Perform some processing
- Return a result (optional)

Explanation

Imagine you are making tea every morning.

The steps are always the same:

1. Boil water
2. Add tea leaves
3. Add milk
4. Add sugar
5. Serve

Instead of remembering all five steps every day, think of them as one task called **makeTea()**.

Similarly, in Java, we group related statements into a method.

Without Methods

```
System.out.println("Welcome");
System.out.println("Have a Nice Day");

System.out.println("Welcome");
System.out.println("Have a Nice Day");

System.out.println("Welcome");
System.out.println("Have a Nice Day");
```

The same code is repeated.

With Methods

```
public static void greet() {
   System.out.println("Welcome");
   System.out.println("Have a Nice Day");
}
```

Calling:

```
greet();
greet();
greet();
```

Output

Welcome
Have a Nice Day
Welcome
Have a Nice Day

Welcome
Have a Nice Day
The code is written **once** but can be used many times.

Why Do We Need Methods?

Methods provide many advantages.

1. Code Reusability

Write once.
Use many times.

2. Reduces Code Duplication

Instead of writing the same statements repeatedly, write them once inside a method.

3. Improves Readability

Compare:

```
calculateSalary();
```

vs.
50 lines of salary calculation.
The method name clearly describes its purpose.

4. Easier Maintenance

If the salary calculation changes, update **one method** instead of every place where the code is repeated.

5. Modular Programming

A large program is divided into smaller, manageable methods.

Syntax of a Method

```
accessModifier returnType methodName(parameters){
   // Method Body
}
```

Example

```
public static void greet(){
   System.out.println("Hello");
}
```

Understanding Each Part

Access Modifier

Controls where the method can be accessed.

Examples

public
private
protected
default

Return Type

Specifies what the method returns.

Example

int

returns an integer.
double
returns a decimal value.
String
returns text.
void
returns nothing.

Method Name
The name chosen by the programmer.
Example
calculateSalary()
printMarks()
findMaximum()

Parameters
Input accepted by the method.
Example
(int a, int b)

Method Body
Contains the statements that perform the task.

Method Declaration

Definition
A **method declaration** specifies the method's name, return type, and parameters without describing what it does.
Example (interface)
```
void display();
```
Only the declaration is provided.

Method Definition

Definition
A **method definition** includes the actual implementation (body) of the method.
Example
```
void display(){
   System.out.println("Hello");
}
```

Calling a Method

Definition
Calling a method means executing the code inside that method.
Example
```
public static void greet(){
   System.out.println("Welcome");
}
public static void main(String[] args){
   greet();
}
```

Output
Welcome

Types of Methods

Methods are commonly classified into four types.

1. No Parameters, No Return Value

```
public static void greet(){
   System.out.println("Hello");
}
```

2. Parameters, No Return Value

```
public static void greet(String name){
   System.out.println("Hello " + name);
}
```

Call

```
greet("Rahul");
```

Output
Hello Rahul

3. No Parameters, Return Value

```
public static int getNumber(){
   return 100;
}
```

4. Parameters and Return Value

```
public static int add(int a,int b){
   return a+b;
}
```

Call

```
int result = add(10,20);
```

Output
30

Parameters

Definition
A **parameter** is a variable declared in the method definition that receives data from the caller.
Example

```
void add(int a,int b)
```

Here,

- a
- b

are parameters.

Arguments

Definition
An **argument** is the actual value supplied when calling a method.
Example

```
add(10,20);
```

Here,

- 10

- 20

Parameter vs Argument

Parameter	Argument
Declared in method	Passed during method call
Placeholder	Actual value
Receives data	Sends data

Example

```
void square(int number)
```

number → Parameter

```
square(5);
```

5 → Argument

Return Statement

Definition

The return statement ends a method and optionally sends a value back to the caller.

Example

```
public static int square(int x){
   return x*x;
}
```

void Method

Definition

A void method performs a task but **does not return any value**.

Example

```
public static void welcome(){
   System.out.println("Welcome");
}
```

Method Overloading

Definition

Method overloading means creating multiple methods with the **same name** but **different parameter lists**.
The compiler determines which method to call based on the number, type, or order of parameters.

Example

```
int add(int a,int b){
   return a+b;
}
double add(double a,double b){
   return a+b;
}
int add(int a,int b,int c){
   return a+b+c;
}
```

This improves readability because one logical operation (add) can work with different inputs.

Pass by Value

Definition

Java is **pass-by-value**.
When a method is called, a **copy** of the argument is passed to the method.

Changing the parameter inside the method **does not change** the original variable.

Example

```
public static void change(int x){
   x = 100;
}
public static void main(String[] args){
   int number = 10;
   change(number);
   System.out.println(number);
}
```

Output

10

The original variable remains unchanged because only a copy was modified.

Variable-Length Arguments (Varargs)

Definition

A **varargs** parameter allows a method to accept **zero or more arguments** of the same type.

Syntax

dataType... parameterName

Example

```
public static int sum(int... numbers){
   int total = 0;
   for(int n : numbers){
      total += n;
   }
  return total;
}
```

Calls

```
sum(10,20);
sum(10,20,30,40);
sum();
```

All are valid.

Recursion

Definition

Recursion is a technique in which a method calls itself to solve a problem.

A recursive method must include a **base case** to stop further calls.

Example

```
public static void countDown(int n){
   if(n == 0){
      return;
   }
   System.out.println(n);
   countDown(n - 1);
}
```

Output for countDown(3):

3

2

1

Method Call Stack

Definition

Whenever a method is called, Java places it on the **call stack**.

When the method finishes, it is removed from the stack.

Example

main()

↓

add()

↓

square()

↓

Return

↓

Return

↓

End

The last method called is the first one to finish (**Last In, First Out – LIFO**).

Scope of Variables

Local Variable

Declared inside a method.

Accessible only within that method.

Example

```
public static void demo(){
   int x = 10;
}
```

x cannot be accessed outside demo().

Parameter Scope

Parameters exist only inside the method.

Example

```
void display(int age)
```

age is accessible only within display().

Common Mistakes

Forgetting to Call the Method

```
public static void greet(){
   System.out.println("Hello");
}
```

If greet(); is never called, nothing is printed.

Missing return

This causes a compile-time error because a non-void method must return a value.

Wrong Number of Arguments

```
add(10);
```

when the method expects two arguments:

```
add(int a,int b)
```

This results in a compilation error.

Advantages of Methods

- Code reuse
- Less duplication
- Easier debugging
- Better readability
- Easier testing
- Better organization
- Supports modular programming

Practice Questions

- What is a method? Why is it used?
- Explain the syntax of a method.
- What is the difference between a method declaration and a method definition?
- What is the difference between parameters and arguments?
- What is the purpose of the return statement?
- What is the difference between a void method and a non-void method?
- Explain method overloading with an example.
- What does "Java is pass-by-value" mean?
- What are varargs? When should you use them?
- What is recursion? Why is a base case important?
- What is the method call stack?
- Explain the scope of local variables and parameters.

Chapter 8: Object-Oriented Programming (OOP) in Java

What is Object-Oriented Programming (OOP)?

Definition

Object-Oriented Programming (OOP) is a programming paradigm that organizes software around **objects** rather than functions. An object contains **data (fields)** and **behavior (methods)**. In Java, almost everything is built around objects.

Explanation

In the real world, everything can be considered an object.

Examples:

- Car
- Mobile Phone
- Student
- Employee
- Bank Account

Every object has:

1. State (Data)

Represents the characteristics of an object.

Example:

A Student has:

- Name
- Roll Number
- Age
- Marks

2. Behavior (Methods)

Represents what the object can do.

Example:

A Student can:

- Study
- Write Exam
- Attend Class

3. Identity

Uniquely identifies one object from another.

Example:

Two students may have the same name but different roll numbers.

Real-Life Example

Car

State

Color

Brand

Speed

Fuel

Behavior

Start()

Stop()

Brake()
Accelerate()
Identity
Registration Number

Why OOP?

OOP solves many problems found in procedural programming.
Advantages:

- Code Reusability
- Better Security
- Easier Maintenance
- Modularity
- Scalability
- Real-world Modeling
- Easy Debugging
- Reduced Code Duplication

Procedural Programming vs Object-Oriented Programming

Procedural Programming	**Object-Oriented Programming**
Focuses on functions	Focuses on objects
Data and functions are separate	Data and methods are together
Less secure	More secure through encapsulation
Difficult to maintain large programs	Easier to maintain
Example: C	Example: Java, C++, C#

What is a Class?

Definition

A **class** is a **blueprint (template)** used to create objects. It defines the **fields (variables)** and **methods (behaviors)** that objects of that class will have.
A class itself does **not** occupy memory for instance data. Memory is allocated only when objects are created.

Syntax

```
class ClassName {
   // Fields
   // Methods
}
```

Example

```
class Student {
   String name;
   int age;
   void study() {
      System.out.println("Studying");
   }
}
```

The class describes what every Student object should contain.

What is an Object?

Definition

An **object** is a real instance of a class. It occupies memory and can access the fields and methods defined by its class.

Example

```
Student s1 = new Student();
```

Here:

- Student → Class
- s1 → Reference variable
- new → Creates a new object
- Student() → Constructor

Memory Representation

Reference Variable

```
s1
 |
 ▼
 ----------------------
| name = null          |
| age = 0              |
|----------------------|
| study()              |
 ----------------------
```

Difference Between Class and Object

Class	Object
Blueprint	Instance of a class
No memory for instance data	Occupies memory
Logical entity	Physical entity
Defines properties and behavior	Uses those properties and behaviors
One class can create many objects	Each object is independent

Creating Objects

Syntax

```
ClassName reference = new ClassName();
```

Example

```
Student student = new Student();
```

Accessing Fields

```
student.name = "Rahul";
student.age = 20;
```

Calling Methods

```
student.study();
```

Fields (Instance Variables)

Definition

A **field** (also called an **instance variable**) is a variable declared inside a class but outside any method. Each object gets its own copy of these variables.

Example

```
class Employee {
   String name;
   double salary;
}
```

Each Employee object has its own name and salary.

Methods in a Class

Methods define the behavior of an object.

Example

```
class Calculator {
   int add(int a, int b) {
      return a + b;
   }
}
```

Constructor

Definition

A **constructor** is a special member of a class that is automatically called when an object is created. Its main purpose is to initialize the object's state.

Characteristics

- Same name as the class
- No return type (not even void)
- Invoked automatically during object creation
- Can be overloaded

Syntax

```
class Student {
   Student() {
   }
}
```

Default Constructor

Definition

If a class does not define any constructor, the Java compiler provides a **default constructor**.

Example

```
class Student {
}
```

Equivalent to

```
class Student {
   Student() {
   }
}
```

User-Defined Constructor

Example

```
class Student {
   Student() {
      System.out.println("Object Created");
   }
}
```

Output

Object Created
when an object is created.

Parameterized Constructor

Definition

A constructor that accepts parameters is called a **parameterized constructor**.

Example

```
class Student {
   String name;
   Student(String name) {
      this.name = name;
   }
}
```

Creating an object

```
Student s = new Student("Rahul");
```

Constructor Overloading

Constructors can have the same name (the class name) but different parameter lists.

Example

```
class Student {
   Student() {
   }
   Student(String name) {
   Student(String name, int age) {
   }
}
```

this Keyword

Definition

The this keyword refers to the **current object**.
It is used to distinguish between instance variables and parameters with the same name, invoke another constructor, or pass the current object.

Example

```
class Student {
   String name;
   Student(String name) {
      this.name = name;
   }
}
```

this.name refers to the instance variable, while name refers to the constructor parameter.

Calling Another Constructor

```
class Student {
  Student() {
    this("Unknown");
  }
  Student(String name) {
    System.out.println(name);
  }
}
```

this(...) must be the **first statement** in the constructor.

static Keyword

Definition

The static keyword makes a member belong to the **class** rather than to individual objects.
There is only one copy of a static member, shared by all objects.

Static Variable

```
class Student {
  static String school = "ABC School";
}
```

All students share the same school name.

Static Method

```
class MathUtil {
  static int square(int n) {
    return n * n;
  }
}
```

Call

```
MathUtil.square(5);
```

A static method belongs to the class, so no object is required.

Difference Between Static and Instance Members

Static	Instance
Belongs to class	Belongs to object
One copy	Separate copy per object
Access using class name	Access using object
Created when class loads	Created when object is created

final Keyword

Definition

The final keyword restricts modification.
Its meaning depends on where it is used.

Final Variable

Cannot be reassigned after initialization.

Example

```
final double PI = 3.14159;
```

Final Method

Cannot be overridden in a subclass.

```
class Animal {
   final void eat() {
   }
}
```

Final Class

Cannot be inherited.

```
final class Utility {
}
```

new Keyword

Definition

The new keyword creates an object and allocates memory for it.

Example

```
Student s = new Student();
```

null Reference

Definition

null indicates that a reference variable does not currently refer to any object.

Example

```
Student s = null;
```

Trying to access members through s will throw a NullPointerException.

Practice Questions

- What is Object-Oriented Programming?
- Explain the concepts of state, behavior, and identity with an example.
- Differentiate between a class and an object.
- What is a constructor? How is it different from a method?
- What is the purpose of the this keyword?
- Explain constructor overloading with an example.
- What is the difference between static and instance members?
- Explain the three uses of the final keyword.
- What does the new keyword do?
- What does null mean in Java?

Chapter 8 : Core Principles of Object-Oriented Programming (OOP)

Four Pillars of OOP

Definition

The **Four Pillars of OOP** are the fundamental principles that make object-oriented programming powerful, reusable, secure, and easy to maintain.

The four pillars are:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

Encapsulation

Definition

Encapsulation is the process of **wrapping data (fields) and methods (behavior) into a single unit (class)** while **restricting direct access** to the data.

It is also known as **Data Hiding**.

Why Encapsulation?

Without encapsulation:

- Anyone can modify important data.
- Invalid values may be stored.
- Programs become less secure.

With encapsulation:

- Data is protected.
- Validation can be performed before changing data.
- Code becomes easier to maintain.

Example

```
class Student {
   private String name;
   public void setName(String name) {
      this.name = name;
   }
   public String getName() {
      return name;
   }
}
Using the class
Student s = new Student();
s.setName("Rahul");
System.out.println(s.getName());
```

Output

Rahul

Getter Method

Definition

A **getter** is a public method used to **read** the value of a private field.

Example

```
public int getAge() {
   return age;
}
```

Setter Method

Definition

A **setter** is a public method used to **update** the value of a private field.

Example

```
public void setAge(int age) {
   if(age >= 0)
      this.age = age;
   }
}
```

Notice that validation is performed before assigning the value.

Advantages of Encapsulation

- Data hiding
- Better security
- Validation of data
- Easier maintenance
- Better code organization

Inheritance

Definition

Inheritance is the mechanism by which one class acquires the properties and methods of another class.
The existing class is called the **parent (superclass)**, and the new class is called the **child (subclass)**.

Syntax

```
class Parent {
}
class Child extends Parent {
}
```

Example

```
class Animal {
   void eat() {
      System.out.println("Eating");
   }
}
class Dog extends Animal {
   void bark() {
      System.out.println("Barking");
   }
}
```

Using the class

```
Dog d = new Dog();
d.eat();
d.bark();
```

Output

Eating
Barking

Advantages

- Code reusability
- Easier maintenance
- Supports method overriding
- Represents real-world relationships

Types of Inheritance

1. Single Inheritance

One child inherits from one parent.

```
Animal
  |
 Dog
```

2. Multilevel Inheritance

```
Animal
  |
Mammal
  |
 Dog
```

3. Hierarchical Inheritance

```
   Animal
   /   \
  Dog   Cat
```

4. Multiple Inheritance

A class inherits from more than one parent.

```
A    B
 \  /
  C
```

Java does NOT support multiple inheritance with classes to avoid ambiguity (Diamond Problem).
However, Java **does support multiple inheritance through interfaces**.

5. Hybrid Inheritance

A combination of different inheritance types.
Java supports hybrid inheritance only through interfaces.

super Keyword

Definition

The super keyword refers to the **immediate parent class**.

Uses of super

1. Access Parent Variables

```
class Animal {
   String color = "White";
}
class Dog extends Animal {
   String color = "Black";
   void printColor() {
      System.out.println(super.color);
   }
}
```

Output

White

2. Call Parent Method

```
class Animal {
   void sound() {
      System.out.println("Animal Sound");
   }
}
class Dog extends Animal {
   void sound() {
      super.sound();
      System.out.println("Bark")
   }
}
```

3. Call Parent Constructor

```
class Animal {
   Animal() {
      System.out.println("Animal Constructor");
   }
}
class Dog extends Animal {
   Dog() {
      super();
   }
}
```

super() must be the **first statement** in a constructor.

Polymorphism

Definition

Polymorphism means **one interface, many forms**.

The same method call can perform different actions depending on the object.

Types

1. Compile-Time Polymorphism
2. Runtime Polymorphism

Compile-Time Polymorphism

Achieved through **method overloading**.

Example

```
class Calculator {
   int add(int a,int b){
      return a+b;
   }
   double add(double a,double b){
     return a+b;
   }
}
```

The compiler decides which method to call.

Runtime Polymorphism

Achieved through **method overriding**.

Example

```
class Animal {
   void sound(){
      System.out.println("Animal Sound");
   }
}
class Dog extends Animal {
   @Override
   void sound(){
      System.out.println("Bark");
   }
}
```

Using

```
Animal a = new Dog();
a.sound();
```

Output

Bark

The method is chosen at **runtime** based on the actual object.

Method Overriding

Definition

Method overriding occurs when a child class provides its own implementation of a method already defined in the parent class.

Rules

- Same method name
- Same parameter list
- Same or covariant return type
- Cannot reduce access level
- Parent method must not be final

Example

```
class Animal {
  void move(){
    System.out.println("Moving");
  }
}
class Bird extends Animal {
  @Override
  void move(){
    System.out.println("Flying");
  }
}
```

Overloading vs Overriding

Overloading	Overriding
Same class	Parent and child classes
Different parameters	Same parameters
Compile-time	Runtime
Increases flexibility	Changes inherited behavior

Dynamic Method Dispatch

Definition

Dynamic Method Dispatch is the mechanism by which Java determines **at runtime** which overridden method to execute.

Example

```
Animal obj = new Dog();
obj.sound();
```

Output

Bark

Even though the reference type is Animal, the actual object is Dog, so Dog's method runs.

Abstraction

Definition

Abstraction means **hiding implementation details and showing only the essential features**.

The user knows **what** an object does, not **how** it does it.

Real-Life Example

When driving a car:

You use

- Steering wheel
- Accelerator
- Brake

You do **not** need to know how the engine works internally.

Abstract Class

Definition

An **abstract class** is a class declared with the abstract keyword.

It cannot be instantiated directly.

Example

```
abstract class Animal {
   abstract void sound();
}
```

Concrete Class

A subclass provides the implementation.

```
class Dog extends Animal {
   @Override
   void sound(){
      System.out.println("Bark");
   }
}
```

Abstract Method

Definition

An **abstract method** has no body and must be implemented by subclasses.

Example

```
abstract void draw();
```

Interface

Definition

An **interface** is a reference type that defines a contract for classes.

A class that implements an interface **must implement** its abstract methods (unless the class itself is abstract).

Syntax

```
interface Vehicle {
   void start();
}
```

Implementation

```
class Car implements Vehicle {
   public void start(){
      System.out.println("Car Started");
   }
}
```

Abstract Class vs Interface

Abstract Class	Interface
Can have constructors	Cannot have constructors
Can have instance variables	Only constants (public static final)
Uses extends	Uses implements
Supports partial implementation	Defines a contract; methods are abstract by default (modern Java also allows default, static, and private methods)
Single inheritance	Multiple interfaces can be implemented

Association

Definition

Association is a relationship where two independent objects work together.

Example

Teacher ---- Student

Both can exist independently.

Aggregation

Definition

Aggregation is a **weak "has-a" relationship**.

The contained object can exist independently of the container.

Example

Departmen

has

Teachers

Teachers still exist even if the department is removed.

Composition

Definition

Composition is a **strong "has-a" relationship**.

The contained object's lifetime depends on the container.

Example

House

has

Rooms

If the house no longer exists conceptually, its rooms are considered part of that house and do not exist independently in that relationship.

Difference Between Aggregation and Composition

Aggregation	Composition
Weak relationship	Strong relationship
Independent lifecycle	Dependent lifecycle
Ownership is shared or loose	Ownership is exclusive
Example: Department–Teacher	Example: House–Room

Advantages of OOP

- Reusability
- Security
- Maintainability
- Flexibility
- Modularity
- Scalability
- Real-world modeling
- Easier testing and debugging

Common Mistakes

Forgetting @Override

```
@Override
void sound() {
}
```

Using @Override helps the compiler detect mistakes in method signatures.

Trying Multiple Class Inheritance

```
class C extends A, B {
}
```

❌ Not allowed in Java.

Creating an Abstract Class Object

```
Animal a = new Animal();
```

❌ Compilation Error.

Abstract classes cannot be instantiated.

Practice Questions

- What are the four pillars of OOP?
- What is encapsulation? Explain with getters and setters.
- Differentiate between encapsulation and abstraction.
- What is inheritance? List its types.
- Why doesn't Java support multiple inheritance with classes?
- Explain the three uses of the super keyword.
- What is polymorphism? Differentiate compile-time and runtime polymorphism.
- What is method overriding? What are its rules?
- What is dynamic method dispatch?
- Differentiate an abstract class and an interface.
- Explain association, aggregation, and composition with examples.
- Compare method overloading and method overriding.

Chapter 9: Exception Handling in Java

What is an Exception?

Definition

An **exception** is an **unexpected event** that occurs during the execution of a program and disrupts the normal flow of instructions.

If an exception is not handled properly, the program terminates abnormally.

Explanation

Suppose a program asks the user to enter a number and then divides 100 by that number.

If the user enters:

0

The program cannot divide by zero.

Instead of continuing normally, Java throws an exception.

Example

```
int a = 100;
int b = 0;

System.out.println(a / b);
```

Output

```
Exception in thread "main"
java.lang.ArithmeticException: / by zero
```

Real-Life Example

Imagine driving a car.

Normal flow:

Start Car

↓

Drive

↓

Reach Destination

Unexpected event:

Tyre Puncture

You must repair the tyre before continuing.

Similarly, an exception interrupts normal program execution.

Why Exception Handling?

Without exception handling:

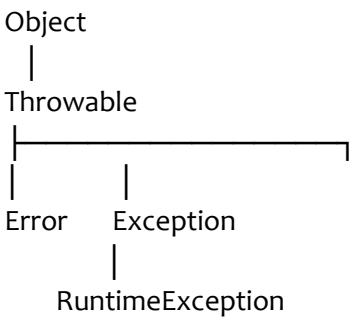
- Program crashes.
- Remaining code is not executed.
- Poor user experience.

With exception handling:

- Program continues safely.
- Errors can be reported clearly.
- Resources are cleaned up properly.

Exception Hierarchy

Java exceptions are organized into a class hierarchy.



Throwable

Definition

Throwable is the **root class** of all errors and exceptions in Java.

It has two direct subclasses:

- Error
- Exception

Error

Definition

An **Error** represents serious problems that applications usually **should not try to handle**.

Examples:

- OutOfMemoryError
- StackOverflowError
- VirtualMachineError

Example

Infinite recursion:

```
public class Main {
  static void test() {
    test();
  }
  public static void main(String[] args) {
    test();
  }
}
```

Eventually produces:

StackOverflowError

Exception

Definition

An **Exception** is a condition that a program can detect and often recover from.

Examples:

- Invalid user input
- Missing file
- Division by zero
- Database connection failure

Types of Exceptions

Java has two major categories.

1. Checked Exceptions
2. Unchecked Exceptions

Checked Exception

Definition

A **checked exception** is checked by the compiler.

The programmer **must** handle it or declare it using throws.

Examples

- IOException
- FileNotFoundException
- SQLException
- ClassNotFoundException

Example

```
FileReader file = new FileReader("data.txt");
```

The compiler requires handling because the file may not exist.

Unchecked Exception

Definition

Unchecked exceptions occur during runtime.

They are **not checked** by the compiler.

Examples

- ArithmeticException
- NullPointerException
- ArrayIndexOutOfBoundsException
- NumberFormatException

Example

```
int x = 10 / 0;
```

Produces

ArithmeticException

Difference Between Checked and Unchecked Exceptions

Checked Exception	Unchecked Exception
Checked by compiler	Checked at runtime
Must be handled or declared	Handling is optional
Subclasses of Exception (excluding RuntimeException)	Subclasses of RuntimeException
Example: IOException	Example: NullPointerException

try Block

Definition

A try block contains code that may throw an exception.

Syntax

```
try {
   // risky code
}
```

Example

```
try {
   int result = 10 / 0;
}
```

catch Block

Definition

A catch block handles an exception thrown inside the try block.

Syntax

```
try {
}
catch(Exception e){
}
```

Example

```
try {
   int result = 10 / 0;
}
catch(ArithmeticException e){
   System.out.println("Cannot divide by zero.");
}
```

Output

Cannot divide by zero.

finally Block

Definition

The finally block contains code that executes **whether an exception occurs or not**.
It is commonly used for cleanup tasks, such as closing files or database connections.

Example

```
try {
   System.out.println("Try");
}
finally {
   System.out.println("Finally");
}
```

Output

Try
Finally

try-catch-finally

```
try {
   int x = 10 / 0;
}
catch(ArithmeticException e){
   System.out.println("Handled");
}
finally{
   System.out.println("Always Executes");
}
```

Output

Handled

Always Executes

Multiple catch Blocks

Definition

A single try block may have multiple catch blocks to handle different exception types.

Example

```
try {
   int[] arr = new int[2];
   System.out.println(arr[5]);
}
catch(ArrayIndexOutOfBoundsException e){
   System.out.println("Invalid Index");
}
catch(Exception e){
   System.out.println("General Exception");
}
```

Order of catch Blocks

Always place **specific exceptions first** and more general exceptions later.

Correct

```
catch(IOException e)
catch(Exception e)
```

Wrong

```
catch(Exception e)
catch(IOException e)
```

The second block becomes unreachable.

Nested try

```
try {
   try {
      int x = 10 / 0;
   }
   catch(ArithmeticException e){
      System.out.println("Inner");
   }
```

```
}
catch(Exception e){
   System.out.println("Outer");
}
```

---

throw Keyword

Definition

The throw keyword is used to **explicitly create and throw an exception.**

Example

```
int age = 15;
if(age < 18){
   throw new ArithmeticException("Not Eligible");
}
```

throws Keyword

Definition

The throws keyword declares that a method may throw one or more exceptions.
The caller is responsible for handling them.

Example

```
void readFile() throws IOException {
}
```

Difference Between throw and throws

throw	throws
Used inside a method	Used in method declaration
Throws one exception object	Declares one or more possible exceptions
Followed by an exception object	Followed by exception class names

Custom Exception

Definition

A **custom exception** is a user-defined exception created by extending the Exception class (or another suitable exception class).

Example

```
class InvalidAgeException extends Exception
   InvalidAgeException(String message){
      super(message);
   }
}
Using
if(age < 18){
   throw new InvalidAgeException("Age must be 18 or above.");
}
```

Exception Propagation

Definition

If a method does not handle an exception, it is passed to the calling method. This process continues up the call stack until the exception is handled or reaches the JVM.

Example

main()
↓
methodA()
↓
methodB()
↓
Exception

Try-with-Resources

Definition

Introduced in Java 7, **try-with-resources** automatically closes resources that implement AutoCloseable.

Example

```
try (FileReader file = new FileReader("data.txt")) {
   // Read file
}
catch(IOException e){
   System.out.println(e.getMessage());
}
```

No explicit close() call is needed.

Common Runtime Exceptions

Exception	Cause
ArithmeticException	Division by zero
NullPointerException	Using a null reference
ArrayIndexOutOfBoundsException	Invalid array index
NumberFormatException	Invalid string-to-number conversion
ClassCastException	Invalid type casting
IllegalArgumentException	Invalid method argument

Best Practices

- Catch the **most specific exception** possible.
- Do not use exceptions for normal program flow.
- Avoid empty catch blocks.
- Always release resources (or use try-with-resources).
- Provide meaningful exception messages.
- Create custom exceptions when appropriate.

Common Mistakes

Empty catch Block

```
catch(Exception e){
}
```
The exception is silently ignored.

Catching Exception Everywhere

```
catch(Exception e)
```
Prefer catching specific exceptions such as:
```
catch(IOException e)
```
or
```
catch(ArithmeticException e)
```

Ignoring Resources

Opening a file without closing it may lead to resource leaks. Prefer try-with-resources.

Practice Questions

- What is an exception?
- Differentiate between Error and Exception.
- What is the difference between checked and unchecked exceptions?
- Explain the purpose of try, catch, and finally.
- What is the purpose of the throw keyword?
- Differentiate between throw and throws.
- What is a custom exception? Why would you create one?
- Explain exception propagation.
- What is try-with-resources? Why is it useful?
- List five common runtime exceptions and explain their causes.

Chapter 10: Packages and Access Modifiers in Java

What is a Package?

Definition

A **package** is a namespace that groups related classes, interfaces, enums, and sub-packages together. It helps organize Java programs and prevents naming conflicts.

In simple words, a package is like a **folder** that stores related Java files.

Real-Life Example

Think of a computer.

Documents

```
│
├── Java
│    ├── Student.java
│    ├── Teacher.java
│
├── Python
│    ├── main.py
```

Similarly, Java packages organize related classes.

Why Do We Need Packages?

Without packages:

- Large projects become difficult to manage.
- Classes with the same name may conflict.
- Code becomes less organized.

With packages:

- Better organization.
- Avoid naming conflicts.
- Easier maintenance.
- Better security through access control.
- Code reuse.

Types of Packages

Java provides two types of packages.

1. Built-in Packages

Provided by Java.

Examples:

- java.lang
- java.util
- java.io
- java.net
- java.sql
- java.time

2. User-Defined Packages

Created by programmers.

Example

com.company.school

Built-in Packages

java.lang

Automatically imported by Java.

Contains commonly used classes.

Examples:

- String
- Math
- System
- Object
- Integer
- Double

Example

```
System.out.println("Hello");
```

No import statement is required.

java.util

Contains utility classes.

Examples:

- Scanner
- ArrayList
- HashMap
- Collections
- Arrays
- Random

Example

```
import java.util.Scanner;
```

java.io

Used for input and output operations.

Examples:

- File
- FileReader
- FileWriter
- BufferedReader

java.time

Used for date and time.

Examples:

- LocalDate
- LocalTime
- LocalDateTime

Creating a Package

Syntax

package packageName;

The package declaration must be the **first non-comment statement** in the source file.

Example

```
package school;
public class Student {
}
```

Directory Structure
school/
  Student.java

Package Naming Conventions

Java package names are usually:

- Written in lowercase.
- Based on a reversed internet domain name.

Examples:
com.google.map
org.apache.commons
com.company.project

Importing Packages

Definition
The import keyword allows you to use classes from another package without writing their fully qualified names every time.

Syntax
import packageName.ClassName;
Example
import java.util.Scanner;

Importing All Classes

Syntax
```
import java.util.*;
```
This imports all public classes in the package.

Example
```
import java.util.*;
ArrayList<String> list = new ArrayList<>();
Random random = new Random();
```

Fully Qualified Class Name

Instead of importing, you can use the complete package name.
Example
java.util.Scanner input = new java.util.Scanner(System.in);
Useful when two packages contain classes with the same name.

Static Import

Definition
The import static statement imports static members so they can be used without the class name.
Without Static Import
```
System.out.println(Math.sqrt(25));
```

With Static Import

```
import static java.lang.Math.*;
System.out.println(sqrt(25));
```

Output

5.0

Access Modifiers

Definition

Access modifiers determine **where a class, method, constructor, or variable can be accessed**.

Java provides four access modifiers:

- public
- private
- protected
- Default (package-private)

public

Definition

A public member is accessible from **any class in any package**.

Example

```
public class Student {
   public void display() {
      System.out.println("Hello");
   }
}
```

private

Definition

A private member is accessible **only within the same class**.

Example

```
class Student {
   private int age;
}
```

Outside access

```
Student s = new Student();
s.age = 20;
```

Compilation Error.

protected

Definition

A protected member is accessible:

- Inside the same class.
- Inside the same package.
- In subclasses (even if they are in another package).

Example

```
protected void display() {
}
```

Default Access (Package-Private)

Definition

If no access modifier is specified, Java uses **default (package-private)** access.

The member is accessible **only within the same package.**

Example

```
class Student {
   void show() {
   }
}
```

Access Modifier Comparison

Modifier	Same Class	Same Package	Subclass (Different Package)	Different Package (Non-Subclass)
private	☑	✗	✗	✗
Default	☑	☑	✗	✗
protected	☑	☑	☑	✗
public	☑	☑	☑	☑

Class Access Modifiers

Top-level classes can use only:

- public
- Default (package-private)

Not allowed:

- private
- protected

Example

```
public class Student {
}
or
class Student {
}
```

Package vs Import

Package	Import
Groups related classes	Allows use of classes from another package
Declared with package	Declared with import
One package declaration per file	Multiple imports allowed

Name Conflicts

Suppose two packages contain a class named Date.

java.util.Date

java.sql.Date

Instead of importing both, use the fully qualified name.

```
java.util.Date utilDate = new java.util.Date();
java.sql.Date sqlDate = new java.sql.Date(System.currentTimeMillis());
```

Advantages of Packages

- Organize code.
- Avoid class name conflicts.
- Improve readability.
- Easier maintenance.
- Better access control.
- Support modular development.

Best Practices

- Use meaningful package names.
- Follow lowercase naming conventions.
- Keep related classes in the same package.
- Avoid wildcard imports in very large projects if explicit imports improve readability.
- Use private whenever possible to protect data.
- Expose only what is necessary using public or protected.

Practice Questions

- What is a package? Why is it needed?
- Differentiate between built-in and user-defined packages.
- Explain the purpose of the import keyword.
- What is a fully qualified class name?
- What is static import? Give an example.
- Explain the four access modifiers.
- Which access modifiers can be used for top-level classes?
- Compare private, protected, default, and public.
- Why do Java package names use lowercase letters?
- What are the advantages of packages?

Chapter 11: Java Collections Framework (JCF)

What is the Java Collections Framework?

Definition

The **Java Collections Framework (JCF)** is a set of **interfaces, classes, and algorithms** provided by Java to store and manipulate groups of objects efficiently.

Instead of creating your own data structures, you can use the classes provided by the Collections Framework.

Explanation

Suppose you want to store:

- 10 student names
- 500 employee records
- 1 million customer IDs

Using arrays can become difficult because arrays have a **fixed size**.

Collections solve this problem by providing **dynamic, flexible, and efficient** data structures.

Real-Life Example

Imagine a library.

Different storage methods are used for different purposes:

- Books arranged on shelves → List
- Unique membership IDs → Set
- Phone directory (Name → Number) → Map
- Waiting line → Queue

Similarly, Java provides different collection classes for different needs.

Why Collections are Needed?

Arrays have several limitations:

- Fixed size.
- Difficult insertion and deletion.
- Limited built-in methods.
- Only length property.

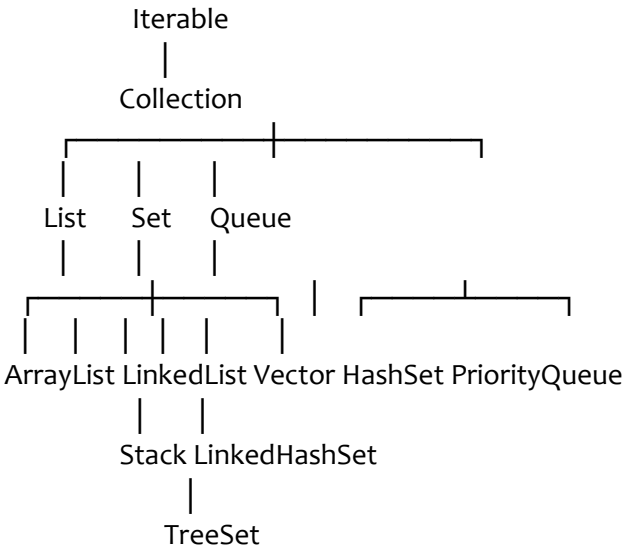
Collections provide:

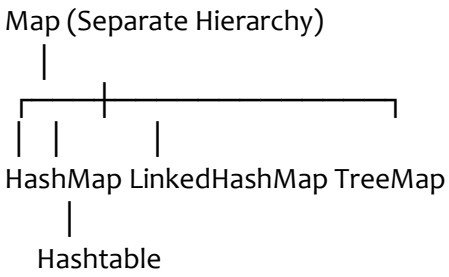
- Dynamic size.
- Easy insertion and deletion.
- Searching.
- Sorting.
- Built-in utility methods.
- Better performance for many operations.

Advantages of Collections

- Dynamic memory allocation.
- Easy data manipulation.
- Reusable classes.
- Built-in algorithms.
- Better code readability.
- Supports Generics.
- Reduces programming effort.

Collection Hierarchy





Note: Map is **not** part of the Collection interface. It is a separate hierarchy.

Collection Interface

Definition

The **Collection** interface is the **root interface** of most collection classes.

It defines common operations such as:

- add()
- remove()
- clear()
- contains()
- size()
- isEmpty()

Common Methods of Collection

Method	Description
add()	Adds an element
remove()	Removes an element
contains()	Checks whether an element exists
size()	Returns number of elements
isEmpty()	Checks if the collection is empty
clear()	Removes all elements

List Interface

Definition

A **List** is an ordered collection that:

- Maintains insertion order.
- Allows duplicate elements.
- Allows positional access using indexes.

Characteristics

- Ordered
- Duplicates allowed
- Index-based
- Dynamic size

Implementations

- ArrayList
- LinkedList
- Vector
- Stack

ArrayList

Definition

ArrayList is a **dynamic array** implementation of the List interface.

Characteristics

- Dynamic size.
- Fast random access.
- Maintains insertion order.
- Allows duplicates.
- Allows multiple null values.
- Not synchronized.

Example

```
import java.util.ArrayList
ArrayList<String> names = new ArrayList<>();
names.add("Alice");
names.add("Bob");
names.add("Alice");
System.out.println(names);
```

Output

[Alice, Bob, Alice]

Common Methods

Method	Description
add()	Add element
get(index)	Get element
set(index, value)	Replace element
remove(index)	Remove element
contains()	Search element

Method	Description
size()	Number of elements

Advantages

- Fast retrieval (get()).
- Dynamic resizing.
- Easy to use.

Disadvantages

- Slow insertion/deletion in the middle because elements must be shifted.

LinkedList

Definition

LinkedList is a doubly linked list implementation of both the List and Deque interfaces.

Characteristics

- Dynamic size.
- Maintains insertion order.
- Allows duplicates.
- Fast insertion and deletion.
- Slower random access.

Example

```
import java.util.LinkedList;
LinkedList<Integer> list = new LinkedList<>();
list.add(10);
list.add(20);
list.add(30);
System.out.println(list);
```

Output

[10, 20, 30]

ArrayList vs LinkedList

ArrayList	LinkedList
Uses dynamic array	Uses doubly linked list
Fast random access	Slow random access
Slow insertion/deletion in middle	Fast insertion/deletion when position is known
Less memory per element	More memory (stores links)

Vector

Definition

Vector is similar to ArrayList, but its methods are **synchronized**, making it thread-safe.

Characteristics

- Dynamic array.
- Thread-safe.
- Slower than ArrayList in single-threaded programs due to synchronization overhead.

- Maintains insertion order.
- Allows duplicates.

Example

```
Vector<String> v = new Vector<>();
v.add("Java");
```

Stack

Definition

Stack is a class that follows the **LIFO (Last In, First Out)** principle.

Real-Life Example

A stack of plates.
The last plate placed on top is removed first.

Operations

- push()
- pop()
- peek()
- empty()
- search()

Example

```
Stack<Integer> stack = new Stack<>();
stack.push(10);
stack.push(20);
System.out.println(stack.pop());
```

Output

```
20
```

Queue

Definition

A **Queue** follows the **FIFO (First In, First Out)** principle.

Real-Life Example

People standing in a ticket line.
The first person who joins the line is served first.

Common Methods

Method	Description
offer()	Insert element
poll()	Remove head
peek()	View head
element()	View head (throws exception if empty)

PriorityQueue

Definition

A PriorityQueue orders elements according to their **priority** (natural ordering or a comparator), not necessarily their insertion order.

Example

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(30);
pq.offer(10);
pq.offer(20);
System.out.println(pq.poll());
```

Output

10

Deque

Definition

A **Deque (Double-Ended Queue)** allows insertion and removal from **both the front and the rear**.

Implementations include:

- ArrayDeque
- LinkedList

Set Interface

Definition

A **Set** is a collection that **does not allow duplicate elements**.

Characteristics

- Unique elements only.
- No index-based access.
- Order depends on implementation.

Implementations

- HashSet
- LinkedHashSet
- TreeSet

HashSet

Definition

HashSet stores **unique elements** using a hash table.

Characteristics

- No duplicates.
- No guaranteed order.
- Allows one null element.
- Fast lookup on average.

Example

```
HashSet<String> set = new HashSet<>();
set.add("Java");
set.add("Python");
set.add("Java");
System.out.println(set);
```

Output (order may vary)

[Java, Python]

LinkedHashSet

Definition

LinkedHashSet maintains **insertion order** while preventing duplicates.

Example

```
LinkedHashSet<Integer> set = new LinkedHashSet<>();
set.add(3);
set.add(1);
set.add(2);
System.out.println(set);
```

Output

[3, 1, 2]

TreeSet

Definition

TreeSet stores unique elements in **sorted order**.

Example

```
TreeSet<Integer> set = new TreeSet<>();
set.add(30);
set.add(10);
set.add(20);
System.out.println(set);
```

Output

[10, 20, 30]

Practice Questions

- What is the Java Collections Framework?
- Why are collections preferred over arrays?
- Explain the Collection hierarchy.
- What is the difference between List and Set?
- Compare ArrayList and LinkedList.
- Why is Vector slower than ArrayList?
- Explain the LIFO principle with Stack.
- Explain the FIFO principle with Queue.
- Compare HashSet, LinkedHashSet, and TreeSet.
- When would you choose a PriorityQueue over a regular queue?

Chapter 11 (Part 2): Maps, Iterators, Generics & Sorting

What is a Map?

Definition

A **Map** is a data structure that stores data in **key-value pairs**.
Each **key** must be **unique**, while **values** can be duplicated.
Unlike List and Set, **Map is not a child of the Collection interface**.

Real-Life Example

Think of a phone book.

Name	Phone Number
Alice	9876543210
Bob	9123456789

- Name → Key
- Phone Number → Value

A person (key) has one associated phone number (value).

Characteristics of Map

- Stores key-value pairs.
- Keys are unique.
- Values can be duplicated.
- Provides fast searching using keys.
- Cannot contain duplicate keys.

Map Hierarchy

```
Map
|
├── HashMap
├── LinkedHashMap
├── TreeMap
└── Hashtable
```

HashMap

Definition

HashMap is the most commonly used implementation of the Map interface.
It stores data using a **hash table**, providing fast average-case lookup, insertion, and deletion.

Characteristics

- Key-value pairs.
- Unique keys.
- Values may repeat.
- No guaranteed order.
- Allows one null key.
- Allows multiple null values.
- Not synchronized.

Example

```
import java.util.HashMap;
HashMap<Integer, String> students = new HashMap<>();
students.put(101, "Alice");
students.put(102, "Bob");
students.put(103, "Charlie");
System.out.println(students);
```

Output (order may vary)

{101=Alice, 102=Bob, 103=Charlie}

Common Methods

Method	Purpose
put()	Add key-value pair
get()	Get value by key
remove()	Remove entry
containsKey()	Check key
containsValue()	Check value
size()	Number of entries
clear()	Remove all entries

LinkedHashMap

Definition

LinkedHashMap maintains the **insertion order** of entries while providing hash table performance.

Example

```
LinkedHashMap<Integer, String> map = new LinkedHashMap<>();
ap.put(3, "C");
map.put(1, "A");
map.put(2, "B");
System.out.println(map);
```

Output

{3=C, 1=A, 2=B}

TreeMap

Definition

TreeMap stores entries in **sorted order of keys**.

Characteristics

- Keys are sorted.
- No null keys.
- Values may be null.
- Slower than HashMap due to sorting.

Example

```
TreeMap<Integer, String> map = new TreeMap<>();
map.put(30, "C");
map.put(10, "A");
```

```
map.put(20, "B");
System.out.println(map);
```

Output

{10=A, 20=B, 30=C}

Hashtable

Definition

Hashtable is a legacy implementation of the Map interface.
It is **thread-safe** because its methods are synchronized.

Characteristics

- Synchronized.
- No null keys.
- No null values.
- Slower than HashMap in single-threaded programs.

HashMap vs Hashtable

HashMap	Hashtable
Not synchronized	Synchronized
Faster	Slower
One null key allowed	No null key
Multiple null values allowed	No null values

Map.Entry

Definition

A Map.Entry represents a single key-value pair in a map.

Example

```
for (Map.Entry<Integer, String> entry : students.entrySet()) {
   System.out.println(entry.getKey() + " : " + entry.getValue());
}
```

Output

101 : Alice
102 : Bob
103 : Charlie

Iterator

Definition

An **Iterator** is an object used to traverse elements of a collection one by one.

Why Use an Iterator?

- Traverse collections safely.
- Remove elements during iteration using remove().
- Works with most collection types.

Method	Purpose
hasNext()	Checks if more elements exist
next()	Returns the next element
remove()	Removes the current element

Example

```
ArrayList<String> names = new ArrayList<>();
names.add("Alice");
names.add("Bob");
Iterator<String> it = names.iterator();
while (it.hasNext()) {
   System.out.println(it.next());
}
```

Output

Alice
Bob

ListIterator

Definition

A **ListIterator** is an enhanced iterator that works only with List implementations.

Features

- Forward traversal.
- Backward traversal.
- Modify elements.
- Add new elements during iteration.

Methods

Method	Purpose
next()	Move forward
previous()	Move backward
add()	Add element
set()	Replace element

Iterator vs ListIterator

Iterator	ListIterator
Forward only	Forward and backward
Read/remove	Read, remove, add, replace
Works with all collections	Works only with lists

Generics

Definition

Generics allow classes, interfaces, and methods to work with different data types while providing **compile-time type safety**.

Without Generics

```
ArrayList list = new ArrayList();
list.add("Alice");
list.add(100);
```
Different types can be mixed, which may cause runtime errors.

With Generics
```
ArrayList<String> list = new ArrayList<>();
list.add("Alice");
```
Only String values are allowed.

Advantages of Generics
- Type safety.
- Compile-time error checking.
- No explicit type casting.
- Cleaner and more readable code.
- Reusable code.

Generic Class
```
class Box<T> {
  private T value;
  public void set(T value) {
    this.value = value;
  }
  public T get() {
    return value;
  }
}
```
Usage
```
Box<String> box = new Box<>();
box.set("Java");
System.out.println(box.get());
```

Generic Method
```
public static <T> void print(T value) {
  System.out.println(value);
}
```

Wildcards
Wildcards are represented by the ? symbol.

Unbounded Wildcard
List<?> list;
Accepts a list of any type.

Upper Bounded Wildcard
List<? extends Number>
Accepts Number and its subclasses.
Examples
- Integer

- Double
- Float

Lower Bounded Wildcard
List<? super Integer>
Accepts Integer or its superclasses.

Comparable Interface

Definition
Comparable defines the **natural ordering** of objects.
A class implements Comparable when it knows how it should be sorted by default.

Method
compareTo()

Example
```
class Student implements Comparable<Student> {
   int marks;
   @Override
   public int compareTo(Student other) {
      return this.marks - other.marks;
   }
}
```

Comparator Interface

Definition
A Comparator provides **custom sorting logic** without changing the class itself.

Example
```
Comparator<Student> byName = (a, b) -> a.name.compareTo(b.name);
```
This sorts students alphabetically by name.

Comparable vs Comparator

Comparable	Comparator
Natural ordering	Custom ordering
Implemented inside the class	Implemented outside the class
One default sorting rule	Multiple sorting rules possible
Uses compareTo()	Uses compare()

Collections Utility Class

Definition
The Collections class provides static utility methods for collection operations.

Common Methods

Method	Purpose
sort()	Sort elements
reverse()	Reverse order

Method	Purpose
shuffle()	Randomize order
max()	Largest element
min()	Smallest element
binarySearch()	Search in sorted list
frequency()	Count occurrences

Example

```
ArrayList<Integer> list = new ArrayList<>();
list.add(30);
list.add(10);
list.add(20);
Collections.sort(list);
System.out.println(list);
```

Output

[10, 20, 30]

Time Complexity (Average Case)

Collection	Search	Insert	Delete
ArrayList	O(n)	O(1)*	O(n)
LinkedList	O(n)	O(1)**	O(1)**
HashSet	O(1)	O(1)	O(1)
TreeSet	O(log n)	O(log n)	O(log n)
HashMap	O(1)	O(1)	O(1)
TreeMap	O(log n)	O(log n)	O(log n)

* Appending at the end is amortized **O(1)**; inserting in the middle is **O(n)**.

** Insertion/deletion is **O(1)** when you already have a reference to the node (or iterator). Finding the position is **O(n)**.

Practice Questions

- What is the difference between Collection and Map?
- Compare HashMap, LinkedHashMap, TreeMap, and Hashtable.
- What is Map.Entry?
- Explain the purpose of an Iterator.
- Differentiate Iterator and ListIterator.
- What are Generics? What are their advantages?
- Explain unbounded, upper-bounded, and lower-bounded wildcards.
- Compare Comparable and Comparator.
- List five useful methods of the Collections utility class.
- Why can removing elements inside a for-each loop cause problems?

Chapter 12: Multithreading in Java

What is a Process?

Definition

A **process** is an independent program in execution. It has its own memory space, resources, and execution environment.

Examples of Processes

- Google Chrome
- Microsoft Word
- Visual Studio Code
- Spotify

Each runs as a separate process.

Characteristics of a Process

- Independent execution.
- Own memory.
- Own resources.
- Communication between processes is relatively expensive.

What is a Thread?

Definition

A **thread** is the **smallest unit of execution** within a process.
A process can contain one or more threads that share the same memory and resources.

Example

Google Chrome:

- UI Thread
- Download Thread
- Rendering Thread
- Network Thread

All belong to the same process.

Process vs Thread

Process	Thread
Independent program	Smallest execution unit
Own memory	Shares process memory
Heavyweight	Lightweight
Creation is slower	Creation is faster
Communication is slower	Communication is faster

What is Multithreading?

Definition

Multithreading is the ability of a program to execute **multiple threads concurrently** within the same process.

Example

A music player can:

- Play music
- Download songs
- Update UI

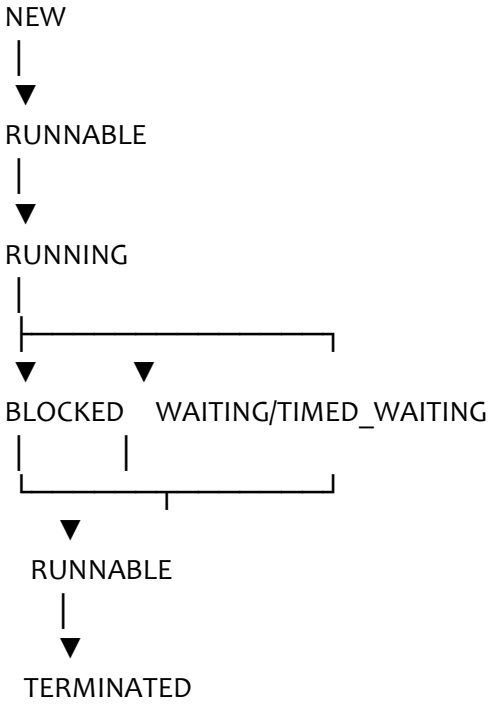
all at the same time.

Advantages

- Better CPU utilization.
- Faster execution.
- Improved responsiveness.
- Efficient resource sharing.
- Parallel task execution (subject to available CPU cores and scheduling).

Thread Life Cycle

A thread goes through several states during execution.



States Explained

NEW

Thread object created.

Thread t = new Thread();

RUNNABLE

start() has been called. The thread is eligible to run.

RUNNING

The scheduler selects the thread, and it executes.

BLOCKED

Waiting to acquire a monitor lock.

WAITING

Waiting indefinitely for another thread to perform an action (for example, after calling wait()).

TIMED_WAITING
Waiting for a specified time.
Example:
Thread.sleep(1000);

TERMINATED
Thread has finished execution.

Creating a Thread

Java provides two common ways.

Method 1: Extending the Thread Class

```
class MyThread extends Thread {
   @Override
   public void run() {
      System.out.println("Thread Running")
   }
}
Starting the thread
MyThread t = new MyThread();
t.start();
```

Method 2: Implementing the Runnable Interface

```
class MyTask implements Runnable {
   @Override
   public void run() {
      System.out.println("Task Running");
   }
}
Running
Thread t = new Thread(new MyTask());
t.start();
```

Thread vs Runnable

Thread	Runnable
Extends Thread	Implements Runnable
Cannot extend another class	Can still extend another class
Less flexible	More flexible
Less preferred	Preferred in most applications

start() vs run()

start()
Creates a **new thread** and then calls run() internally.
t.start();

run()
Acts like a normal method call if invoked directly.

t.run();
No new thread is created.

Difference

start()	run()
Creates new thread	No new thread
Executes concurrently	Executes in current thread
Calls run() internally	Just a normal method

Thread Methods

sleep()
Definition
Pauses the current thread for a specified time.
Example
```
Thread.sleep(1000);
```
Sleeps for approximately one second.

join()
Definition
Makes the current thread wait until another thread finishes.
Example
```
Thread t = new Thread()
t.start();
t.join();
```

yield()
Definition
Hints to the scheduler that the current thread is willing to let other runnable threads execute.
Example
```
Thread.yield();
```
The scheduler may ignore this hint.

Thread Priority

Each thread has a priority.
Range
1 to 10
Constants
```
Thread.MIN_PRIORITY     //
Thread.NORM_PRIORITY    // 5
Thread.MAX_PRIORITY     // 10
Setting priority
t.setPriority(8);
```
Thread priority is only a scheduling hint. It does not guarantee execution order.

Daemon Thread

Definition
A **daemon thread** runs in the background and supports user threads.
Examples:

- Garbage Collector
- Background monitoring

Creating
t.setDaemon(true);
This must be called **before** start().

User Thread vs Daemon Thread

User Thread	Daemon Thread
Performs application work	Supports background tasks
Keeps JVM alive	JVM may exit when only daemon threads remain
Example: Main thread	Example: Garbage Collector

Synchronization

Definition
Synchronization controls access to shared resources so that only one thread can execute a critical section at a time.

Why Synchronization?
Without synchronization:
Two threads may modify shared data simultaneously.
This can lead to inconsistent results.

Race Condition
Definition
A **race condition** occurs when multiple threads access and modify shared data concurrently, and the result depends on the timing of execution.

Example
Suppose:
Balance = 1000
Thread A withdraws 200.
Thread B withdraws 300.
Without synchronization, the final balance may become incorrect due to overlapping updates.

synchronized Keyword
Definition
The synchronized keyword ensures that only one thread at a time can execute a synchronized method or block using the same monitor.

Synchronized Method

```
class Counter
   private int count;
   synchronized void increment() {
      count++;
   }
}
```

Synchronized Block

```
synchronized(this) {
   count++;
}
```

This locks only the specified block instead of the whole method.

Inter-Thread Communication

Java provides:

- wait()
- notify()
- notifyAll()

These methods belong to the Object class and must be called while holding the object's monitor (inside synchronized code).

wait()

Temporarily releases the monitor and waits until notified (or interrupted/timed out if using a timed version).

notify()

Wakes one waiting thread.

notifyAll()

Wakes all waiting threads.

Deadlock

Definition

A **deadlock** occurs when two or more threads wait indefinitely for each other to release resources.

Example

Thread
Needs Lock B
Thread B
Needs Lock A
Neither thread can continue.

Causes

- Circular waiting.
- Multiple locks acquired in inconsistent order.

Prevention

- Acquire locks in a consistent order.
- Reduce lock scope.
- Avoid unnecessary nested locks.

Thread Safety

Definition

A class is **thread-safe** if it behaves correctly when accessed by multiple threads simultaneously.

Ways to Achieve Thread Safety

- Synchronization.

- Immutable objects.
- Concurrent collections.
- Atomic classes.

Executor Framework

Definition

The **Executor Framework** (introduced in Java 5) manages thread creation and execution efficiently using thread pools.

Example

```
ExecutorService service = Executors.newFixedThreadPool(3)
service.submit(() -> System.out.println("Task"));
service.shutdown();
```

Advantages:

- Reuses threads.
- Better performance.
- Easier task management.

Common Thread Methods

Method	Purpose
start()	Starts a new thread
run()	Contains thread task
sleep()	Pause current thread
join()	Wait for another thread
yield()	Hint scheduler
isAlive()	Check if thread is still running
setName()	Set thread name
getName()	Get thread name
setPriority()	Set priority
currentThread()	Get current thread

Practice Questions

- What is the difference between a process and a thread?
- What is multithreading? List its advantages.
- Explain the thread life cycle.
- Compare extending Thread with implementing Runnable.
- What is the difference between start() and run()?
- Explain sleep(), join(), and yield().
- What is a daemon thread?
- What is synchronization? Why is it needed?
- What is a race condition?
- What is deadlock? How can it be prevented?
- What is thread safety?
- Why is the Executor Framework preferred in modern Java?

Chapter 13: File Handling (Java I/O & NIO)

What is File Handling?

Definition

File Handling is the process of creating, reading, writing, updating, deleting, and managing files stored on a computer using Java programs.

Java provides two main APIs:

- **Java I/O (java.io)** – Traditional stream-based file handling.
- **Java NIO (java.nio)** – Modern, faster, and more flexible file handling introduced in Java 7.

Why File Handling?

Without file handling:

- Data is lost when the program ends.
- Information cannot be stored permanently.

With file handling:

- Data is stored permanently.
- Programs can read existing files.
- Data can be shared between applications.

Real-Life Examples

- Saving student records.
- Storing game progress.
- Writing application logs.
- Reading configuration files.
- Exporting reports.

Streams

Definition

A **Stream** is a flow of data between a program and a file (or another input/output source).

There are two main types:

- Input Stream
- Output Stream

Input Stream

Reads data **from** a source.

File ----------> Program

Output Stream

Writes data **to** a destination.

Program ----------> File

Types of Streams

Java provides two categories of streams.

1. Byte Streams

Used for binary data.

Examples:

- Images
- Videos

- PDFs
- Audio files

Classes:

- InputStream
- OutputStream
- FileInputStream
- FileOutputStream

2. Character Streams

Used for text data.

Classes:

- Reader
- Writer
- FileReader
- FileWriter

Byte Stream Hierarchy

InputStream
|
FileInputStream

OutputStream
|
FileOutputStream

Character Stream Hierarchy

Reader
|
FileReader

Writer
|
FileWriter

File Class

Definition

The File class represents files and directories.
It does **not** read or write file contents. It is mainly used to manage file information.

Common Constructors

```
File file = new File("data.txt");
File folder = new File("Documents");
```

Common Methods

Method	Purpose
exists()	Checks if file exists
createNewFile()	Creates file
delete()	Deletes file

Method	Purpose
mkdir()	Creates one directory
mkdirs()	Creates nested directories
isFile()	Checks if it is a file
isDirectory()	Checks if it is a directory
length()	Returns file size
renameTo()	Renames file
listFiles()	Lists files in a directory

Example

```
import java.io.File;
File file = new File("student.txt");
System.out.println(file.exists());
```

FileOutputStream

Definition

FileOutputStream writes **binary data** to a file.

Example

```
import java.io.FileOutputStream;
FileOutputStream fos = new FileOutputStream("data.txt");
fos.write(65);
fos.close();
```

Output:

A

(65 is the ASCII value of 'A'.)

FileInputStream

Definition

FileInputStream reads **binary data** from a file.

Example

```
import java.io.FileInputStream;
FileInputStream fis = new FileInputStream("data.txt");
int ch = fis.read()
System.out.println((char) ch);
fis.close();
```

Output

A

FileWriter

Definition

FileWriter writes **character (text)** data to a file.

Example

```
import java.io.FileWriter;
FileWriter writer = new FileWriter("hello.txt");
```

```
writer.write("Welcome to Java");
writer.close();
```

Append Mode

FileWriter writer = new FileWriter("hello.txt", true);

The second argument true appends to the file instead of overwriting it.

FileReader

Definition

FileReader reads **character (text)** data from a file.

Example

```
FileReader reader = new FileReader("hello.txt")
int ch;
while ((ch = reader.read()) != -1)
   System.out.print((char) ch)
}
reader.close();
```

Buffered Streams

Definition

Buffered streams improve performance by reducing direct disk access.

BufferedReader

```
BufferedReader br = new BufferedReader(new FileReader("data.txt"));
String line = br.readLine();
br.close();
```

BufferedWriter

```
BufferedWriter bw = new BufferedWriter(new FileWriter("data.txt"));
bw.write("Hello");
bw.close();
```

Advantages

- Faster reading and writing.
- Efficient for large files.
- Supports reading text line by line (BufferedReader).

Serialization

Definition

Serialization is the process of converting an object into a byte stream so that it can be stored in a file or transmitted over a network.

A class must implement the Serializable interface.

Example

```
class Student implements Serializable {
   int id;
   String name;
}
```

ObjectOutputStream

```
ObjectOutputStream out = new ObjectOutputStream(
   new FileOutputStream("student.dat"));
out.writeObject(student);
out.close();
```

Deserialization

Definition

Deserialization is the process of reconstructing an object from its serialized byte stream.

ObjectInputStream

```
ObjectInputStream in = new ObjectInputStream(
   new FileInputStream("student.dat"));
Student s = (Student) in.readObject();
in.close();
```

Serialization vs Deserialization

Serialization	Deserialization
Object → Bytes	Bytes → Object
ObjectOutputStream	ObjectInputStream
Saving object	Reading object

Java NIO

Definition

Java NIO (New I/O) provides a modern API for file operations.
It is generally more flexible and offers convenient utility methods.

Important Classes

- Path
- Paths
- Files

Creating a Path

```
Path path = Path.of("data.txt");
```

(You may also see Paths.get("data.txt") in older code.)

Reading All Lines

```
List<String> lines = Files.readAllLines(path);
```

Writing

```
Files.writeString(path, "Hello Java");
```

Common Files Methods

Method	Purpose
exists()	Check existence
createFile()	Create file

Method	Purpose
delete()	Delete file
copy()	Copy file
move()	Move file
readAllLines()	Read all lines
readString()	Read entire file
writeString()	Write a string
size()	File size

Try-with-Resources

Definition

Automatically closes resources after use.

Example

```
try (FileReader reader = new FileReader("data.txt")) {
   int ch;
   while ((ch = reader.read()) != -1) {
      System.out.print((char) ch);
  }
}
catch (IOException e) {
   e.printStackTrace();
}
```

No explicit close() is required.

Common File Operations

Create File

```
File file = new File("demo.txt");
file.createNewFile();
```

Delete File

```
file.delete();
```

Check File Exists

```
file.exists();
```

Rename File

```
file.renameTo(new File("newName.txt"));
```

Create Folder

```
new File("Documents").mkdir();
```

Common Exceptions in File Handling

Exception	Cause
IOException	General I/O problem
FileNotFoundException	File does not exist or cannot be opened

Exception	Cause
EOFException	End of file reached unexpectedly
ClassNotFoundException	During deserialization if the class cannot be found

Java I/O vs Java NIO

Java I/O	Java NIO
Stream-based	Path/Channel/Buffer-oriented APIs with convenient Files utilities
Older API	Modern API
Suitable for basic tasks	More flexible and feature-rich
More manual coding	Many ready-made utility methods

Best Practices

- Always close files, or use try-with-resources.
- Prefer character streams for text.
- Use byte streams for binary data.
- Handle exceptions properly.
- Prefer Files utility methods for many common operations.
- Use buffering for better performance on larger files.

Common Mistakes

Forgetting to Close Streams

```
FileWriter writer = new FileWriter("a.txt");
writer.write("Hello");
```

Without close() (or try-with-resources), data may not be fully written.

Using Byte Streams for Text

Possible, but character streams (Reader/Writer) are generally a better choice for text because they handle character encoding more appropriately.

Ignoring Exceptions

```
catch(Exception e) {
}
```

Avoid swallowing exceptions silently.

Practice Questions

- What is file handling?
- Explain input and output streams.
- Differentiate byte streams and character streams.
- What is the purpose of the File class?
- Compare FileInputStream and FileReader.
- What are buffered streams? Why are they faster?
- Explain serialization and deserialization.
- What is Java NIO? How is it different from Java I/O?
- What is the benefit of try-with-resources?
- List common file handling exceptions and their causes.